

cumple, entonces la tarea que invoca la función se quedará bloqueada hasta que se cumpla, es decir, hasta que la barrera se *levante*.

Listado 4.6: Implementación de la tarea Buffer (limitado) en Ada 95.

```

1  -- Buffer de un MaxTamanyo elementos.
2  task Buffer is
3      entry Escribir (Elem: TElemento);
4      entry Leer (Elem: out TElemento);
5  end Buffer;
6
7  task body Buffer is
8      MaxTamanyo: constant := 10; -- Buffer limitado.
9      Datos: array (1..MaxTamanyo) of TElemento;
10     I, J: Integer range 1..MaxTamanyo := 1;
11     Tamanyo: Integer range 0..MaxTamanyo := 0;
12 begin
13     loop
14         select
15             when Tamanyo < MaxTamanyo =>
16                 accept Escribir (Elem: TElemento) do
17                     Datos(i) := Elem; -- Guarda el elemento.
18                 end Escribir;
19                 I := I mod MaxTamanyo + 1;
20                 Tamanyo := Tamanyo + 1;
21             or
22             when Tamanyo > 0 =>
23                 accept Leer (Elem: out TElemento) do
24                     Elem := Datos(J); -- Devuelve el elemento.
25                 end Leer;
26                 J := J mod MaxTamanyo + 1;
27                 Tamanyo := Tamanyo - 1;
28             end select;
29         end loop;
30 end Buffer;
```

En este contexto, estas construcciones son más naturales que el uso de semáforos contadores o buzones de mensajes con un determinado tamaño máximo.

De nuevo, el control concurrente sobre las operaciones *leer* y *escribir* se gestiona mediante el uso de *entries*.

4.2.2. Los objetos protegidos

Hasta ahora se ha estudiado cómo las tareas permiten encapsular datos sin la necesidad de un mecanismo de exclusión mutua explícito. Por ejemplo, en el caso del buffer limitado de la anterior sección, no es necesario utilizar ningún tipo de mecanismo de exclusión mutua ya que su contenido nunca será accedido de manera concurrente. Este hecho se debe a la propia naturaleza de los *entries*.

Sin embargo, utilizar tareas para *servir* datos puede introducir una sobrecarga en la aplicación. Además, las tareas no representan la forma más natural de encapsular datos y, de manera simultánea, ofrecer una funcionalidad a potenciales clientes.



Las regiones críticas condicionales representan un mecanismo de sincronización más natural y sencillo de entender e implementar.

En Ada, un objeto protegido es un tipo de **módulo protegido** que encapsula una estructura de datos y exporta subprogramas o funciones [3]. Estos subprogramas operan sobre la estructura de datos bajo una exclusión mutua automática. Del mismo modo que ocurre con las tareas, en Ada es posible definir barreras o guardas en las propias entradas, que deben evaluarse a verdadero antes de que a una tarea se le permita *entrar*. Este planteamiento se basa en la definición de **regiones críticas condicionales**.

El siguiente listado de código muestra un ejemplo sencillo de tipo protegido para un entero compartido [3].

Listado 4.7: Ejemplo simple de tipo protegido.

```

1 -- Tipo protegido para un entero compartido.
2 protected type EnteroCompartido (ValorInicial : Integer) is
3   function Leer return Integer;
4   procedure Escribir (NuevoValor : Integer);
5   procedure Incrementar (Incremento : Integer);
6
7 private
8   Dato : Integer := ValorInicial;
9
10 end EnteroCompartido;
11
12 V : EnteroCompartido(7);

```

La declaración de V en la línea [13] declara una instancia de dicho tipo protegido y le asigna el valor inicial al dato que éste encapsula. Este valor sólo puede ser accedido mediante *Leer*, *Escribir* e *Incrementar*. En este contexto, es importante destacar las diferencias existentes al usar procedimientos o funciones protegidas:



Las llamadas a entradas protegidas se usan para implementar la sincronización de condición.

- Un **procedimiento protegido**, como *Escribir* o *Incrementar*, proporciona un acceso mutuamente excluyente tanto de lectura como de escritura a la estructura de datos encapsulada por el tipo protegido. En otras palabras, las llamadas a dichos procedimientos se ejecutarán en exclusión mutua.
- Una **función protegida**, como *Leer*, proporciona acceso concurrente de sólo lectura a la estructura de datos encapsulada. En otras palabras, es posible que múltiples tareas ejecuten *Leer* de manera concurrente. No obstante, las llamadas a funciones protegidas son mutuamente excluyentes con las llamadas a procedimientos protegidos.

Finalmente, una **entrada protegida** es similar a un procedimiento protegido. La diferencia reside en que la entrada está asociada a una barrera integrada en el propio objeto protegido. Como se comentó anteriormente, si la evaluación de dicha barrera obtiene un valor lógico *falso*, entonces la tarea que invoca dicha entrada se quedará suspendida hasta que la barrera se evalúe a *verdadero* y, adicionalmente, no existan otras tareas activas dentro del objeto protegido.



Las barreras se evalúan siempre que una nueva tarea evalúa la barrera y ésta haga referencia a una variable que podría haber cambiado desde que la barrera fue evaluada por última vez. Así mismo, las barreras también se evalúan si una tarea abandona un procedimiento o *entry* y existen tareas bloqueadas en barreras asociadas a una variable que podría haber cambiado desde la última evaluación.

A continuación se discutirá un problema de sincronización clásico, ya discutido en la sección 2.3.3, implementado haciendo uso de objetos protegidos.

Modelando el problema de los filósofos comensales

En esta sección se discute una posible implementación del problema de los filósofos comensales utilizando Ada 95 y objetos protegidos².

En la sección 2.3.3, los palillos se definían como recursos indivisibles y se modelaban mediante semáforos binarios. De este modo, un

²Código fuente adaptado a partir de la implementación de http://es.wikibooks.org/wiki/Programaci%C3%B3n_en_Ada/Tareas/Ejemplos

palillo cuyo semáforo tuviera un valor de 1 podría ser utilizado por un filósofo, simulando su obtención a través de un decremento (primitiva *wait*) del semáforo y bloqueando al filósofo adyacente en caso de que este último quisiera cogerlo.

En esta solución, el palillo se modela mediante un **objeto protegido**. Para ello, el siguiente listado de código define el tipo protegido *Cubierto*.

Listado 4.8: Declaración del objeto protegido Cubierto.

```

1 -- Código adaptado de...
2 -- http://es.wikibooks.org/wiki/Programaci
3 package Cubiertos is
4
5     type Cubierto is limited private;
6
7     procedure Coger(C: in out Cubierto);
8     procedure Soltar(C: in out Cubierto);
9
10 private
11
12     type Status is (LIBRE, OCUPADO);
13
14     protected type Cubierto(Estado_Cubierto: Status := LIBRE) is
15         entry Coger;
16         entry Soltar;
17
18     private
19         Estado: Status := Estado_Cubierto;
20     end Cubierto;
21
22 end Cubiertos;
```

Como se puede apreciar, dicho tipo protegido mantiene el estado del cubierto como un elemento de tipo *Status* (línea 13), que puede ser LIBRE u OCUPADO, como un dato privado, mientras que ofrece dos *entries* protegidos: *Coger* y *Soltar* (líneas 16-17). Note cómo el estado del tipo *Cubierto* se inicializa a LIBRE (línea 15).

Por otra parte, el objeto protegido *Cubierto* está integrado dentro del paquete *Cubiertos*, cuya funcionalidad está compuesta por los procedimientos *Coger* y *Soltar* (líneas 8-9). Dichos procedimientos actúan sobre los cubiertos de manera individual (observe el parámetro de entrada/salida *C*, de tipo *Cubierto* en ambos).

A continuación se muestra la implementación de los procedimientos y *entries* declarados en el anterior listado. Note cómo los procedimientos simplemente delegan en los *entries* asociados la gestión de la exclusión mutua. Por ejemplo, el procedimiento público *Coger* del paquete cubiertos invoca el *entry Coger* del cubierto pasado como parámetro a dicho procedimiento (líneas 6-9).

Listado 4.9: Implementación del objeto protegido Cubierto.

```

1 -- Código adaptado de...
2 -- http://es.wikibooks.org/wiki/Programaci
3 package body Cubiertos is
4
5     procedure Coger (C: in out Cubierto) is
6     begin
7         C.Coger;
8     end Coger;
9
10    procedure Soltar (C: in out Cubierto) is
11    begin
12        C.Soltar;
13    end Soltar;
14
15    protected body Cubierto is
16
17        entry Coger when Estado = LIBRE is
18        begin
19            Estado := OCUPADO;
20        end Coger;
21
22
23        entry Soltar when Estado = OCUPADO is
24        begin
25            Estado := LIBRE;
26        end Soltar;
27
28    end Cubierto;
29
30 end Cubiertos;

```

El aspecto más destacable de esta implementación es el uso de sendas barreras en los *entries* *Coger* y *Soltar* para controlar si un cubierto está libre u ocupado antes de cogerlo o soltarlo, respectivamente.

La vida de un filósofo

Recuerde que, en el problema clásico de los filósofos comensales, éstos viven un ciclo continuo compuesta de las actividades de *comer* y *pensar*.

En este punto, el objeto protegido *Cubierto* ya gestiona el acceso concurrente y exclusivo a los distintos cubiertos que se vayan a instanciar. Este número coincide con el número de filósofos que se sentarán a la mesa. Por lo tanto, el siguiente paso consiste en modelar la tarea *Filósofo* y plantear el código necesario para llevar a cabo la simulación del problema.

El siguiente listado de código muestra el modelado de un filósofo. La parte más importante está representada por el procedimiento *Pensar* (líneas [11-18](#)), en el que el filósofo intenta obtener los palillos que se encuentran a su izquierda y a su derecha, respectivamente, antes de comer. El procedimiento *Pensar* (líneas [20-24](#)) simplemente realiza una espera de 3 segundos.

Listado 4.10: Tarea filósofo en Ada.

```

1 -- Código adaptado de...
2 -- http://es.wikibooks.org/wiki/Programacion_en_Ada/Tareas/Ejemplos
3 type PCubierto is access Cubierto;
4
5 task type TFilosofo(Id: Character;
6                   Cubierto_Izquierda: PCubierto; Cubierto_Derecha:
7                   PCubierto);
8
9 task body TFilosofo is
10
11   procedure Comer is
12   begin
13     -- Intenta obtener el cubierto izquierdo y el derecho.
14     Coger(Cubierto_Izquierda.all);
15     Coger(Cubierto_Derecha.all);
16     Put(Id & "c "); delay 1.0;
17     Soltar(Cubierto_Derecha.all);
18     Soltar(Cubierto_Izquierda.all);
19   end Comer;
20
21   Procedure Pensar is
22   begin
23     Put(Id & "p ");
24     delay 3.0;
25   end Pensar;
26 begin
27   loop
28     Comer; Pensar;
29   end loop;
30 end TFilosofo;

```

Finalmente, el siguiente listado muestra la lógica necesaria para simular el problema de los filósofos comensales. Básicamente, lleva a cabo la instanciación de los cubiertos como objetos protegidos (líneas 25-27) y la instanciación de los filósofos (líneas 30-35).

Listado 4.11: Simulación de los filósofos.

```

1 -- Código adaptado de...
2 -- http://es.wikibooks.org/wiki/Programaci
3 with Ada.Text_IO; use Ada.Text_IO;
4 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
5 with Cubiertos; use Cubiertos;
6
7 procedure Problema_Filosofos is
8
9   -- Aquí el código del listado anterior...
10
11   N_Cubiertos: Positive;
12
13 begin
14
15   Put("Número de filósofos: "); Get(N_Cubiertos); New_line;
16
17   declare
18     type PTFilosofo is access TFilosofo;
19     P: PTFilosofo;
20     C: Character := 'A';
21     Cuberteria: array (1..N_Cubiertos) of PCubierto;
22 begin

```

```

23      -- Instanciación de los cubiertos.
24      for i in 1..N_Cubiertos loop
25      Cuberteria(i) := new Cubierto;
26      end loop;
27
28      -- Instanciación de los filósofos (A, B, C, D, etc).
29      for i in 1..N_Cubiertos-1 loop
30      P := new TFilosofo(C, Cuberteria(i), Cuberteria(i+1));
31      C := Character'Succ(C); -- Sucesor para generar el siguiente
        ID.
32      end loop;
33      -- El último filósofo comparte palillo con el primero.
34      P := new TFilosofo(C, Cuberteria(1), Cuberteria(N_Cubiertos))
        ;
35  end;
36 end Problema_Filosofos;

```

Si desea llevar a cabo una simulación para comprobar que, efectivamente, el acceso concurrente a los palillos es correcto, recuerde que ha de ejecutar los siguientes comandos:

```

$ gnatmake problema_filosofos.adb
$ ./problema_filosofos

```

Si la compilación y la ejecución fueron correctas, entonces el resultado obtenido de la simulación debería ser similar al siguiente.

```

Introduzca el numero de cubiertos/filosofos: 5

Ac Cc Ap Ec Cp Bc Ep Dc Bp Dp Ac Cc Ap Ec Cp Bc Ep Dc Bp Dp Ac Cc Ap
Ec Cp Bc Ep Dc Bp Dp Ac Cc Ap Ec Cp Bc Dc Ep Bp Dp Ac Cc Ap Cp Ec Bc
Ep Dc Bp Dp Ac Cc Ap Cp Bc Ec Bp Ep Dc Dp Ac Cc Ap Cp Bc Ec Bp Ep Dc
Dp Ac Cc Ap Cp Bc Ec Bp Ep Dc Dp Ac Cc Cp Ap Ec Bc Ep Bp Dc Dp Cc Ac
Cp Ap Bc Ec Bp Ep Dc Dp Cc Ac Cp Ap Ec Bc Ep Bp Dc Dp Cc Ac Ap Ec Cp
Bc Ep Bp Dc Dp Ac Cc Ap Bc Cp Ec Bp Ep Dc Dp Ac Cc Ap Cp Bc Ec Ep Bp
Dc Dp Cc Ac Cp Ap Bc Ec Bp Dc Ep Dp Cc Ac Cp Ap Bc Ec Bp Ep Dc Dp Cc
Ac Cp Ap Bc Ec Bp Ep Dc Dp Cc Ac Cp Ap Bc Ec Ep Bp Dc

```

donde la primera letra de cada palabra representa el identificador del filósofo (por ejemplo **A**) y la segunda representa el estado (**p**ensando o **c**omiendo). Note cómo el resultado de la simulación no muestra dos filósofos adyacentes (por ejemplo A y B o B y C) comiendo de manera simultánea.

4.3. El concepto de monitor

Con el objetivo principal de mejorar y flexibilizar los mecanismos de sincronización básicos estudiados, uso de semáforos y de colas de mensajes, han surgido herramientas de sincronización de más alto nivel, como los monitores. En esta sección se discutirá el caso concreto de los monitores, introducidos previamente en la sección 1.2.3.

Desde un punto de vista general, un monitor se puede definir como un **tipo abstracto de datos** con un conjunto de operaciones, especificadas por el programador, que operan en exclusión mutua sobre el propio monitor. En otras palabras, se persigue que el acceso a los

elementos del tipo abstracto de datos sea correcto desde el punto de vista de la concurrencia.

Según [3], los monitores surgen como un refinamiento de las secciones críticas condicionales, es decir, como una evolución de las mismas pero con la idea de solventar algunas de las limitaciones discutidas en la sección 4.1. En concreto, la dispersión de las secciones críticas en un programa con relativa complejidad destruye el concepto de programación estructurada, obteniendo así un código cuya legibilidad y mantenimiento se ven afectados.

En la sección 2.3.1 se estudió el uso de semáforos para modelar una solución al **problema clásico del buffer limitado**. En esencia, dicha solución contemplaba el uso de dos semáforos de tipo contador: i) *empty*, para controlar el número de huecos vacíos, y ii) *full*, para controlar el número de huecos llenos. Además, se hacía uso de un semáforo binario, denominado *mutex*, para controlar el acceso a la sección crítica.

Este planteamiento es *poco natural* en el sentido de que no se trata al buffer limitado como una estructura que gestiona el acceso concurrente por parte de otros procesos o hilos. Por el contrario, dicho acceso se controla desde una *perspectiva externa* mediante el uso de diversos semáforos.



El uso de monitores permite encapsular el estado o variables, que serán accedidas en exclusión mutua, del propio monitor.

En este contexto, el monitor permite modelar el buffer de una manera más natural mediante un módulo independiente de manera que las llamadas concurrentes para insertar o extraer elementos del buffer se ejecutan **en exclusión mutua**. En otras palabras, el monitor es, por definición, el responsable de ejecutarlas secuencialmente de manera correcta. La figura 4.1 muestra la sintaxis de un monitor. En la siguiente sección se discutirá cómo se puede utilizar un monitor haciendo uso de las facilidades que proporciona el estándar POSIX.

Sin embargo, y aunque el monitor proporciona exclusión mutua en sus operaciones, existe la necesidad de la sincronización dentro del mismo. Una posible opción para abordar esta problemática podría consistir en hacer uso de semáforos. No obstante, las implementaciones suelen incluir primitivas que sean más sencillas aún que los propios semáforos y que se suelen denominar **variables de condición**.

Las variables de condición, sin embargo, se manejan con primitivas muy parecidas a las de los semáforos, por lo que normalmente se denominan ***wait*** y ***signal***. Así, cuando un proceso o hilo invoca *wait* sobre una variable de condición, entonces se quedará bloqueado hasta que otro proceso o hilo invoque *signal*. Note que *wait* siempre bloqueará al proceso o hilo que lo invoque, lo cual supone una diferencia importante con respecto a su uso en semáforos.


```
monitor nombre_monitor {  
    /* Declaración de variables compartidas */  
    procedure p1 (...) { ... }  
    procedure p2 (...) { ... }  
    ...  
    procedure pn (...) { ... }  
    inicialización (...) {  
        ...  
    }  
}
```

Figura 4.1: Sintaxis general de un monitor.

Al igual que ocurre con los semáforos, una variable de condición tiene asociada alguna estructura de datos que permite almacenar información sobre los procesos o hilos bloqueados. Típicamente, esta estructura de datos está representada por una cola FIFO (*first-in first-out*).

Tenga en cuenta que la liberación de un proceso bloqueado en una variable de condición posibilita que otro proceso pueda acceder a la sección crítica. En este contexto, la primitiva *signal* es la que posibilita este tipo de sincronización. Si no existe ningún proceso bloqueado en una variable de condición, entonces la ejecución de *signal* sobre la misma no tiene ningún efecto. Esta propiedad también representa una diferencia importante con respecto a la misma primitiva en un semáforo, ya que en este último caso sí que tiene un efecto final (incremento del semáforo).



La construcción del monitor garantiza que, en cualquier momento de tiempo, sólo haya un proceso activo en el monitor. Sin embargo, puede haber más procesos o hilos suspendidos en el monitor.

Antes de pasar a la siguiente sección, en la que se estudiará el caso particular de los monitores en POSIX (mediante el uso de *mutexes* y variables de condición), es importante destacar que las variables de condiciones padecen algunas de las **desventajas** asociadas al uso de semáforos, introducidas en la sección 4.1 del presente capítulo. Por ejemplo, el uso de las primitivas *wait* y *signal* puede propiciar errores cometidos por parte del programador y que, en ocasiones, implica que el proceso de depuración para localizarlos sea tedioso.

4.3.1. Monitores en POSIX

Para modelar el comportamiento de un monitor utilizando las primitivas que ofrece el estándar POSIX y, en concreto, la biblioteca **Pthreads**, es necesario utilizar algún mecanismo para garantizar la ex-

clusión mutua en las operaciones del monitor y proporcionar la sincronización necesario. En este contexto, POSIX *Pthreads* ofrece tanto **mutexes** como **variables de condición** para llevar a cabo dicho modelado.

Con el objetivo de ejemplificar el uso de monitores mediante estos dos elementos definidos en POSIX, en esta sección se retoma el ejemplo del buffer limitado previamente discutido en la sección 2.3.1, en la que se hizo uso de semáforos para controlar el acceso al buffer. Para ello, se asumirán las siguientes suposiciones:

1. El buffer estará implementado en base a la definición de monitor.
2. Los productores y consumidores de información, es decir, las entidades que interactúan con el buffer, estarán implementados mediante hilos a través de la biblioteca *Pthread*.

Con el objetivo de encapsular el estado del buffer, siguiendo la filosofía planteada por el concepto de monitor, se ha definido la clase *Buffer* mediante el lenguaje de programación C++, tal y como se muestra en el siguiente listado de código.

Listado 4.12: Clase Buffer.

```

1  #ifndef __BUFFER_H__
2  #define __BUFFER_H__
3
4  #include <pthread.h>
5
6  #define MAX_TAMANYO 5
7
8  class Buffer {
9  public:
10     Buffer ();
11     ~Buffer ();
12     void anyadir (int dato);
13     int extraer ();
14
15 private:
16     int _n_datos;
17     int _primero, _ultimo;
18     int _datos[MAX_TAMANYO];
19
20     pthread_mutex_t _mutex;
21     pthread_cond_t _buffer_no_lleno;
22     pthread_cond_t _buffer_no_vacio;
23 };
24
25 #endif

```

La funcionalidad relevante de la clase *Buffer* está especificada en las líneas 12-13, en las que se declaran las funciones *anyadir* y *extraer*. Respecto al estado del propio buffer, cabe destacar **dos grupos de elementos**:

- Las variables de clase relacionadas con el número de elementos en el buffer y los apuntadores al primer y último elemento (líneas

[16-18]), respectivamente. La figura 4.2 muestra de manera gráfica dichos apuntadores.

- Las variables de clase que realmente modelan el comportamiento del buffer, es decir, el *mutex* de tipo *pthread_mutex_t* para garantizar el acceso excluyente sobre el buffer y las variables de condición para llevar a cabo la sincronización.

Estas **variables de condición** permiten modelar, a su vez, dos situaciones esenciales a la hora de interactuar con el buffer:

1. La imposibilidad de añadir elementos cuando el buffer está lleno. Esta situación se contempla mediante la variable *_buffer_no_lleno* del tipo *pthread_cond*.
2. La imposibilidad de extraer elementos cuando el buffer está vacío. Esta situación se contempla mediante la variable *_buffer_no_vacio* del tipo *pthread_cond*.

El **constructor** y el **destructor** de la clase *Buffer* permiten inicializar el estado de la misma, incluyendo los elementos propios de la biblioteca *Pthread*, y liberar recursos, respectivamente, tal y como se muestra en el siguiente listado de código.

La **función *anyadir*** permite insertar un nuevo elemento en el buffer de tamaño limitado. Con el objetivo de garantizar que dicha función se ejecute en exclusión mutua, el *mutex* se adquiere justo en la primera instrucción (línea [6]) y se libera en la última (línea [20]), como se aprecia en el siguiente listado.

Listado 4.13: Clase Buffer. Constructor y destructor.

```

1  #include "buffer.h"
2  #include <assert.h>
3
4  Buffer::Buffer ():
5      _n_datos(0), _primero(0), _ultimo(0)
6  {
7      pthread_mutex_init(&_mutex, NULL);
8      pthread_cond_init(&_buffer_no_lleno, NULL);
9      pthread_cond_init(&_buffer_no_vacio, NULL);
10 }
11
12 Buffer::~Buffer ()
13 {
14     pthread_mutex_destroy(&_mutex);
15     pthread_cond_destroy(&_buffer_no_lleno);
16     pthread_cond_destroy(&_buffer_no_vacio);
17 }

```

A continuación, y antes de insertar cualquier elemento en el buffer, es necesario comprobar que éste no esté lleno. Para ello, se usa la condición del bucle *while* (línea [8]). Si es así, es decir, si el buffer ha alcanzado su límite, entonces se ejecuta un *wait* sobre la variable de condición *_buffer_no_lleno*, provocando las dos siguientes consecuencias:

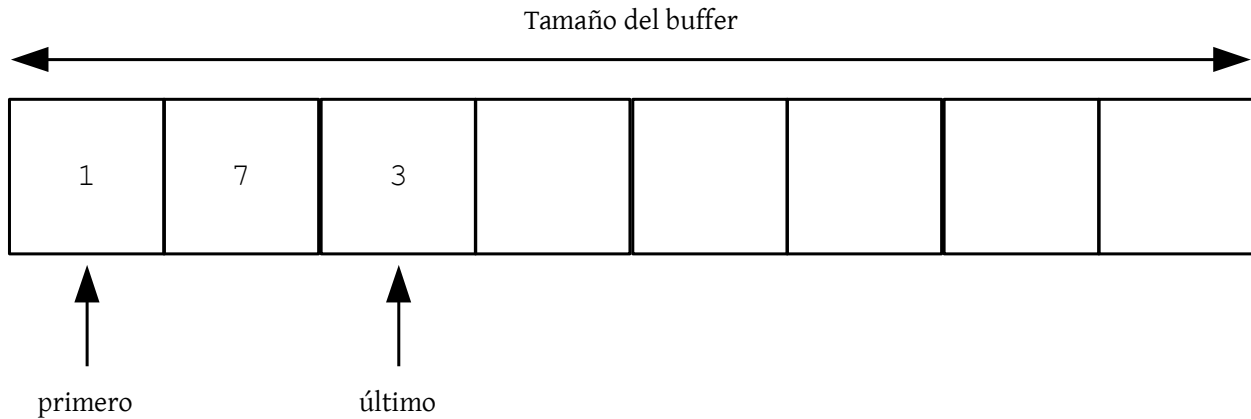


Figura 4.2: Esquema gráfico del buffer limitado modelado mediante un monitor.

1. El hilo que ejecuta *anyadir* se queda bloqueado y se encola en la cola asociada a dicha variable de condición.
2. El *mutex* se libera, con el objetivo de que otro hilo, productor o consumidor, pueda ejecutar *anyadir* o *extraer*, respectivamente. Note cómo el segundo parámetro de *pthread_cond_wait* es precisamente el *mutex* del propio buffer. Así, con un monitor es posible suspender varios hilos **dentro de la sección crítica**, pero sólo es posible que uno se encuentre activo en un instante de tiempo.

Listado 4.14: Clase Buffer. Inserción de elementos.

```

1 void
2 Buffer::anyadir
3 (int dato)
4 {
5     /* Adquisición el mutex para exclusión mutua */
6     pthread_mutex_lock(&mutex);
7     /* Para controlar que no haya desbordamiento */
8     while (_n_datos == MAX_TAMANYO)
9         /* Sincronización */
10        pthread_cond_wait(&buffer_no_lleno, &mutex);
11
12    /* Actualización del estado del buffer */
13    _datos[_ultimo % MAX_TAMANYO] = dato;
14    ++_ultimo;
15    ++_n_datos;
16
17    /* Sincronización */
18    pthread_cond_signal(&buffer_no_vacio);
19
20    pthread_mutex_unlock(&mutex);
21 }

```

La actualización del estado del buffer mediante la función *anyadir* consiste en almacenar el dato en el buffer (línea 13), actualizar el apuntador para el siguiente elemento a insertar (línea 14) e incrementar el número de elementos del buffer (línea 15).

Finalmente, es necesario ejecutar *signal* sobre la variable de condición *_buffer_no_vacio* (línea 18) para desbloquear a algún hilo consumidor bloqueado, si lo hubiera, en dicha variable. Más adelante se comprobará que un hilo consumidor se bloqueará cuando no haya ningún elemento que extraer del buffer. La última instrucción de *anyadir* está asociada a la liberación del *mutex* (línea 20).

El siguiente listado de código muestra la otra función relevante del buffer: **extraer**. Note cómo el esquema planteado es exactamente igual al de *anyadir*, con las siguientes salvedades:

1. Un hilo se suspenderá cuando no haya elementos o datos en el buffer (líneas 9-11) a través de la variable de condición *_buffer_no_vacio*.
2. La actualización de datos implica incrementar el apuntador al primer elemento a extraer y decrementar el número total de datos (líneas 15-16).
3. La ejecución de *signal* se realiza sobre *_buffer_no_lleno*, con el objetivo de desbloquear a un potencial productor de información.
4. El valor extraído se devuelve después de liberar el *mutex*. De otro modo, éste no podría ser adquirido por otro hilo que intente acceder al buffer con posterioridad.

Listado 4.15: Clase Buffer. Extracción de elementos.

```

1  int
2  Buffer::extraer ()
3  {
4      int dato;
5
6      /* Adquisición el mutex para exclusión mutua */
7      pthread_mutex_lock(&_mutex);
8      /* No se puede extraer si no hay nada */
9      while (_n_datos == 0)
10         /* Sincronización */
11         pthread_cond_wait(&_buffer_no_vacio, &_mutex);
12
13     /* Actualización del estado del buffer */
14     dato = _datos[_primero % MAX_TAMANYO];
15     ++_primero;
16     --_n_datos;
17
18     /* Sincronización */
19     pthread_cond_signal(&_buffer_no_lleno);
20
21     pthread_mutex_unlock(&_mutex);
22
23     return dato;
24 }
```

En este punto, con el monitor ya implementado, el siguiente paso es crear un sencillo programa en el que una serie de hilos productores y consumidores, implementados mediante la biblioteca *Pthread*,

interactúen con el monitor (buffer). El siguiente listado muestra la declaración de las funciones asociadas a dichos hilos y la creación de los mismos.

Note cómo la **primitiva *pthread_create*** se utiliza para asociar código a los hilos productores y consumidor, mediante las funciones *productor_func* (línea [21]) y *consumidor_func* (línea [23]), respectivamente. El propio puntero al buffer se pasa como parámetro en ambos casos para que los hilos puedan interactuar con él. Cabe destacar igualmente el uso de la primitiva *pthread_join* (línea [27]) con el objetivo de esperar a los hilos creados anteriormente.

Listado 4.16: Creación de hilos mediante *Pthread*.

```
1 #include <iostream>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5 #include "buffer.h"
6
7 #define NUM_HILOS 8
8
9 static void *productor_func (void *arg);
10 static void *consumidor_func (void *arg);
11
12 int main (int argc, char *argv[]) {
13     Buffer *buffer = new Buffer;
14     pthread_t tids[NUM_HILOS];
15     int i;
16     srand((int) getpid());
17
18     /* Creación de hilos */
19     for (i = 0; i < NUM_HILOS; i++)
20         if ((i % 2) == 0)
21             pthread_create(&tids[i], NULL, productor_func, buffer);
22         else
23             pthread_create(&tids[i], NULL, consumidor_func, buffer);
24
25     /* Esperando a los hilos */
26     for (i = 0; i < NUM_HILOS; i++)
27         pthread_join(tids[i], NULL);
28
29     delete buffer;
30
31     return EXIT_SUCCESS;
32 }
```

Finalmente, el siguiente listado muestra la implementación de los hilos productor y consumidor, respectivamente, mediante sendas funciones. En ellas, estos hilos interactúan con el buffer insertar o extrayendo elementos de manera aleatoria y considerando una breve espera entre operación y operación.

Listado 4.17: Código fuente de los hilos productor y consumidor.

```
1 static void *productor_func (void *arg) {
2     Buffer *buffer = (Buffer *)arg;
3     int i = 0, valor;
4
5     while (i < 10) {
6         sleep(rand() % 3);
7         valor = rand() % 10;
8         buffer->anyadir(valor);
9         std::cout << "Productor:\t" << valor << std::endl;
10        ++i;
11    }
12    return (void *)true;
13 }
14
15 static void *consumidor_func (void *arg) {
16     Buffer *buffer = (Buffer *)arg;
17     int i = 0, valor;
18
19     while (i < 10) {
20         sleep(rand() % 5);
21         valor = buffer->extraer();
22         std::cout << "Consumidor:\t" << valor << std::endl;
23         ++i;
24     }
25     return (void *)true;
26 }
```

4.4. Consideraciones finales

Como ya se introdujo al principio del presente capítulo, concretamente en la sección 4.1, algunas de las limitaciones asociadas a los mecanismos de sincronización de más bajo nivel han propiciado la aparición de otros esquemas de más alto nivel que facilitan la implementación de soluciones concurrentes.

En estos esquemas se encuadran lenguajes de programación como Ada, pero también existen bibliotecas de más alto nivel que incluyen tipos abstractos de datos orientados a la programación concurrente. Con el objetivo de discutir, desde el punto de vista práctico, otra solución distinta a la biblioteca *Pthreads*, el anexo C describe **la biblioteca de hilos de ZeroC ICE**, un *middleware* de comunicaciones orientado a objetos para el desarrollo de aplicaciones cliente/servidor, y los mecanismos de sincronización que ésta proporciona. En este contexto, el lector puede apreciar, desde un punto de vista práctico, cómo las soluciones estudiadas a nivel de proceso en otros capítulos se pueden reutilizar a la hora de interactuar con hilos haciendo uso del **lenguaje de programación C++**.

Finalmente, algunos de los problemas clásicos de sincronización ya estudiados, como por ejemplo los filósofos comensales o el problema del productor/consumidor, se resuelven haciendo uso de la biblioteca de hilos de ICE. Como se podrá apreciar en dicho anexo, las soluciones de más alto nivel suelen mejorar la legibilidad y la mantenibilidad del código fuente.



5

Capítulo

Planificación en Sistemas de Tiempo Real

La programación concurrente está asociada inherentemente a la ejecución en paralelo de procesos o hilos. Con el objetivo de evitar una gestión incorrecta de secciones críticas, el programador hace uso de primitivas de sincronización que proporcionen una exclusión mutua en determinadas partes del código. De este modo, y aunque el comportamiento general de un sistema concurrente sea no determinista, el funcionamiento del mismo es correcto.

No obstante, en determinadas situaciones o sistemas de cómputo es necesario establecer restricciones más fuertes con el objetivo de limitar ese *no determinismo* mencionado anteriormente. Por ejemplo, podría ser deseable limitar el tiempo máximo de respuesta de un proceso ya que, por ejemplo, una respuesta demasiado tardía podría ser innecesaria o incluso no permitida considerando los requisitos del problema a resolver.

En este capítulo se aborda la problemática de la planificación, en el contexto particular de los sistemas de tiempo real, como mecanismo general para establecer un determinismo en la ejecución de programas concurrentes. Para ello, se introduce la noción de *tiempo real* y, a continuación, se discuten algunos esquemas representativos de planificación en tiempo real.

5.1. Introducción

La ejecución de tareas, procesos o hilos está asociada, desde un punto de vista general, al i) núcleo de un sistema operativo, como por ejemplo Linux, o a ii) un entorno de ejecución asociado a un lenguaje concreto, como por ejemplo Ada o Java.

Desde una perspectiva general, en ambos casos se persigue un **aumento de la productividad**. Como ejemplo particular, considere un sistema operativo multi-tarea instalado en un *smartphone* con un procesador que está compuesto de dos núcleos físicos de ejecución. En este contexto, será posible ejecutar de manera paralela, por ejemplo, un navegador web y el despliegue de la agente del teléfono. De este modo, la productividad aumenta aunque el tiempo medio de respuesta se vea afectado.

El resultado de la ejecución concurrente de diversos programas está asociado a un modelo no determinista, en el sentido de que no será posible determinar a priori la secuencia de ejecución de los programas involucrados tras múltiples ejecuciones de los mismos. En otras palabras, no existe a priori una garantía de que el proceso p_i se ejecute en un determinado Δt .

Desafortunadamente, en sistemas con **restricciones temporales** existe una **necesidad de determinismo** que condiciona la propia ejecución de los procesos involucrados. Por ejemplo, considere el sistema de eyección en un avión de combate y la necesidad del piloto de abandonar el avión ante una situación de extrema emergencia. Evidentemente, el tiempo de respuesta de dicho sistema ha de estar acotado por un umbral máximo.

En situaciones de este tipo, con una naturaleza temporal más restrictiva, suele ser necesario utilizar algún tipo de herramienta o incluso lenguaje de programación que dé soporte a este control temporal. En la siguiente sección se discute la noción de *tiempo real* en un contexto de programación concurrente.

5.2. El concepto de *tiempo real*

Tradicionalmente, la noción de *tiempo real* ha estado vinculada a sistemas empujados como aspecto esencial en su funcionamiento. Como ejemplo clásico, considere el sistema de *airbag* de un coche. Si se produce un accidente, el sistema ha de garantizar que empleará como máximo un tiempo Δt . Si no es así, dicho sistema será totalmente ineficaz y las consecuencias del accidente pueden ser catastróficas.

Hasta ahora, y dentro del contexto de una asignatura vinculada a la Programación Concurrente y Tiempo Real, este tipo de restricciones no se ha considerado y se ha pospuesto debido a que, normalmente, el control del tiempo real suele estar construido sobre el modelo de concurrencia de un lenguaje de programación [3].



Figura 5.1: Actualmente, el concepto de multi-tarea, unido a la proliferación de procesadores multi-núcleo, es esencial para incrementar la productividad de los sistemas software.

Herramienta adecuada

Benjamin Franklin llegó a afirmar que si tuviera que emplear 8 horas en talar un árbol, usaría 6 para afilar el hacha. Elija siempre la herramienta más adecuada para solucionar un problema. En el contexto de la programación en tiempo real, existen entornos y lenguajes de programación, como por ejemplo Ada, que proporcionan un gran soporte al desarrollo de este tipo de soluciones.

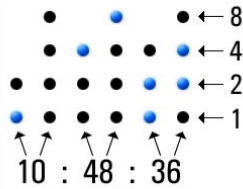


Figura 5.2: El control del tiempo es un aspecto esencial en el ámbito de la programación en tiempo real.

En el ámbito del uso de un lenguaje de programación, la noción de tiempo se puede describir en base a tres elementos independientes [3]:

- **Interfaz con el tiempo**, vinculada al uso de **mecanismos de representación** de conceptos temporales. Por ejemplo, y como se discutirá en breve, considere el uso de una estructura de datos de alto nivel para manejar retardos o *delays*.
- **Representación de requisitos temporales**. Por ejemplo, considere la necesidad de establecer un tiempo máximo de respuesta para un sistema empujado.
- **Satisfacción de requisitos temporales**, vinculada tradicionalmente a la planificación de procesos. Típicamente, la satisfacción de dicho requisitos considerará el comportamiento del sistema en el peor caso posible con el objetivo de calcular una cota superior.



La notación *Landau* $O(f(x))$ se refiere al conjunto de funciones que acotan superiormente a $f(x)$. Es un planteamiento pesimista que considera el comportamiento de una función en el peor caso posible. En el ámbito de la planificación en tiempo real se sigue esta filosofía.

5.2.1. Mecanismos de representación

Relojes

Dentro de un contexto temporal, una de los aspectos esenciales está asociado al **concepto de reloj**. Desde el punto de vista de la programación, un programa tendrá la necesidad de obtener información básica vinculada con el tiempo, como por ejemplo la hora o el tiempo que ha transcurrido desde la ocurrencia de un evento. Dicha funcionalidad se puede conseguir mediante dos caminos distintos:

- Accediendo al marco temporal del entorno.
- Utilizando un reloj hardware externo que sirva como referencia.

La primera opción no es muy común debido a que el propio entorno necesitaría, por ejemplo, llevar a cabo una sincronización con el reloj a través de interrupciones. La segunda opción es más plausible desde la perspectiva del programador, ya que la información temporal se puede obtener mediante las primitivas proporcionadas por un lenguaje de programación o incluso a través del controlador de un reloj interno o externo.

Esta segunda opción puede parecer muy sencilla y práctica, pero puede no ser suficiente en determinados contextos. Por ejemplo,

la programación de sistemas distribuidos puede plantear ciertas complicaciones a la hora de manejar una única referencia temporal por varios servicios. Por ejemplo, considere que los servicios que conforman el sistema distribuido se ejecutan en tres máquinas diferentes. En esta situación, la referencia temporal de dichos servicios será diferente (aunque probablemente mínima). Para abordar esta problemática, se suele definir un *servidor de tiempo* que proporcione una referencia temporal común al resto de servicios. En esencia, el esquema se basa en la misma solución que garantiza un reloj interno o externo, es decir, se basa en garantizar que la información temporal esté sincronizada.



En sistemas POSIX, la referencia temporal se mide en segundos desde el 1 de Enero de 1970.

Retomando el reloj como referencia temporal común, es importante resaltar diversos aspectos asociados al mismo, como la resolución o precisión del mismo (por ejemplo 1 segundo o 1 microsegundo), el rango de valores que es capaz de representar y la exactitud o estabilidad que tiene.

Desde el punto de vista de la programación, cada lenguaje suele proporcionar su propia **abstracción de reloj**. Por ejemplo, Ada proporciona dos paquetes vinculados a dicha abstracción:

- *Calendar*, paquete de biblioteca obligatorio que proporciona estructuras de datos y funciones básicas relativas al tratamiento del tiempo.
- *Real_Time*, paquete de biblioteca opcional que proporciona un nivel de granularidad mayor, entre otras aspectos, que *Calendar*.

El **paquete *Calendar* en Ada** implemente el tipo abstracto de datos *Time*, el cual representa un reloj para leer el tiempo. Así mismo, dicho tipo mantiene una serie de funciones para efectuar conversiones entre elementos de tipo *Time* y otras unidades de tiempo asociadas a tipos del lenguaje, como el tipo *entero* usado para representar años, meses o días, o el tipo *Duration* usado para representar segundos. Por otra parte, *Time* también proporciona operadores aritméticos que sirven para combinar parámetros de tipo *Duration* y *Time* y para comparar valores de tipo *Time*.

El tipo abstracto de datos *Time* es un ejemplo representativo de herramienta, proporcionada en este caso por el lenguaje de programación Ada, para facilitar el tratamiento de información temporal.

A continuación se muestra un listado de código que permite medir el tiempo empleado en llevar a cabo una serie de cálculos. Como se puede apreciar, *Clock* permite obtener referencias temporales (líneas [5] y [8]) y es posible utilizar el operador de resta para obtener un valor de tipo *Duration* al restar dos elementos del tipo *Time* (línea [10]).

Listado 5.1: Midiendo el tiempo en Ada 95.

```
1 declare
2   T_Inicial, T_Final : Time;
3   Intervalo : Duration;
4 begin
5   T_Inicial := Clock;
6   -- Aquí llamadas a funciones
7   -- y otros cálculos.
8   T_Final := Clock;
9   -- Uso de operador -
10  Intervalo := T_Final - T_Inicial;
11 end
```

Por otra parte, el **paquete *Real_Time de Ada*** es similar a *Calendar* pero con algunas diferencias reseñables. Por ejemplo, mantiene un nivel de granularidad mayor, hace uso de una constante *Time_Unit*, que es la cantidad menor de tiempo representada por *Time*, y el rango de *Time* ha de ser de al menos 50 años. En resumen, este paquete representa un nivel de resolución mayor con respecto a *Calendar*.

Retardos

El reloj representa el elemento básico a utilizar por los procesos para poder obtener información temporal. Sin embargo, otro aspecto esencial reside en la capacidad de un proceso, tarea o hilo para suspenderse durante un determinado instante de tiempo. Esta situación está asociada al concepto de **retardo** o **retraso**. Desde un punto de vista general, los retardos pueden clasificarse en dos tipos:

- **Relativos**, los cuales permiten la suspensión respecto al instante actual. Por ejemplo, es posible definir un retardo relativo de 2 segundos.
- **Absolutos**, los cuales permiten retrasar la reanudación de un proceso hasta un instante de tiempo absoluto. Por ejemplo, podría ser necesario garantizar que una acción ha de ejecutarse 5 segundos después del comienzo de otra acción.

En Ada, un retardo relativo se puede implementar fácilmente mediante un bucle de manera que, en cada iteración, el programa compruebe si han transcurrido los n segundos asociados al retardo. El siguiente listado de código muestra un posible ejemplo asociado a un retraso relativo de 7 segundos.

Listado 5.2: Ejemplo de retardo relativo en Ada.

```
1 Inicio := Clock;
2
3 loop
4   exit when (Clock - Inicio) > 7.0;
5 end loop;
```

El principal problema de este planteamiento es la *espera activa*, concepto discutido inicialmente en la sección 1.2.2. En este contexto,

algunos lenguajes de programación incluyen primitivas que eviten este problema. Por ejemplo, Ada introduce la sentencia *delay* para efectuar un retardo relativo:

Listado 5.3: Uso de *delay* en Ada.

```
1 delay 7.0;
```

En POSIX, los retardos relativos se pueden implementar a través de primitivas como *sleep* o *nanosleep*, en función del nivel de granularidad deseado por el programador. Estas primitivas garantizan que el proceso que las invocó será ejecutable después de que transcurra el periodo de tiempo indicado como argumento.



La primitiva *sleep* suspende el hilo de ejecución que realiza la llamada hasta que pasa el tiempo indicado o hasta que recibe una señal no ignorada por el programa. No olvide contemplar este último caso en sus programas.

Por otra parte, un retardo absoluto tiene una complejidad añadida en el sentido de que puede ser necesario calcular el periodo a retrasar o, en función del lenguaje de programación utilizado, emplear una primitiva adicional.

Considere la situación en la que una acción ha de tener lugar 7 segundos después del comienzo de otra. El siguiente fragmento de código en Ada plantea una posible solución.

Listado 5.4: Intento de retraso absoluto en Ada.

```
1 Accion_1; -- Ejecución de Acción 1.  
2 delay 7.0;  
3 Accion_2; -- Ejecución de Acción 2.
```

Este primer planteamiento sería incorrecto, debido a que los 7 segundos de espera se realizan después de la finalización de *Accion_1*, sin considerar el instante de comienzo de la ejecución de ésta.

El siguiente listado contempla esta problemática. Como se puede apreciar, la variable *Transcurrido* de la línea [3] almacena el tiempo empleado en ejecutar *Accion_1*. De este modo, es posible realizar un retraso de $7,0 - \text{Transcurrido}$ segundos (línea [5]) antes de ejecutar *Accion_2*.

Desafortunadamente, este planteamiento puede no ser correcto ya que la instrucción de la línea [5] no es atómica, es decir, no se ejecuta en un instante de tiempo indivisible. Por ejemplo, considere la situación en la que *Accion_1* consume 2 segundos. En este caso en concreto, $7,0 - \text{Transcurrido}$ debería equivaler a 5,0. Sin embargo, si justo después de realizar este cálculo se produce un cambio de contexto, el retraso se incrementaría más allá de los 5,0 segundos calculados.

Listado 5.5: Retraso absoluto en Ada.

```

1 Inicio := Clock;
2 Accion_1; -- Ejecución de Acción 1.
3 Transcurrido := Clock - Inicio;
4
5 delay 7.0 - (Transcurrido);
6
7 Accion_2; -- Ejecución de Acción 2.

```

Algunos lenguajes de programación introducen sentencias específicas que permiten gestionar este tipo de retardos absolutos. En el caso particular de Ada, esta sentencia está representada por la construcción *delay until* <Time>:

Note cómo *delay until* establece un límite inferior, el propio retraso absoluto, antes de que *Accion_2* se ejecute. No obstante, esta solución no contempla la posibilidad de que *Accion_1* tarde más de 7 segundos en ejecutarse.

Listado 5.6: Uso de delay until en Ada.

```

1 Inicio := Clock;
2 Accion_1; -- Ejecución de Acción 1.
3
4 delay until (Inicio + 7.0);
5
6 Accion_2; -- Ejecución de Acción 2.

```

Tanto en retardos absolutos como en retardos relativos, el tiempo sobrepasado se denomina **deriva local** y no es posible eliminarlo [3]. Sin embargo, es posible combatir este problema eliminando la **deriva acumulada** que se produce cuando las derivas locales se superponen.

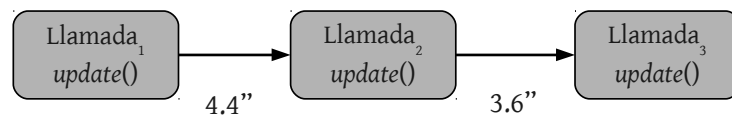


Figura 5.3: Ejemplo de control de la deriva acumulada. Las llamadas a *update* se realizan cada 4 segundos de media.

Por ejemplo, considere que es necesario realizar una llamada a una función *update* cada 4 segundos de media para evitar la deriva local que se vaya acumulando (ver figura 5.3). El siguiente listado de código muestra una posible solución utilizando Ada.

Como se puede apreciar, la sentencia *delay until* combinada con la actualización de *SiguienteLlamada* (líneas 12-13) permite eliminar la deriva acumulada y realizar llamadas a *update* cada 4 segundos.

Listado 5.7: Gestión de la deriva acumulada en Ada.

```

1 declare
2   SiguienteLlamada : Time;
3   -- Tiempo medio de actualización.
4   Intervalo : constant Duration 5.0;
5
6 begin
7   SiguienteLlamada := Clock + Intervalo;
8
9   loop
10    Update;
11    -- Retraso absoluto.
12    delay until SiguienteLlamada;
13    SiguienteLlamada := SiguienteLlamada + Intervalo;
14  end loop;
15
16 end;
```

Finalmente, resulta interesante destacar algunos factores relevantes que afectan a los retardos y que hay que considerar:

- La granularidad del retardo y la del reloj pueden ser distintas. Por ejemplo, POSIX soporta una granularidad hasta el nivel de los nanosegundos, mientras que es posible que el reloj de algún sistema no llegue hasta dicho nivel.
- El propio reloj interno puede verse afectado si se implementa mediante un mecanismo basado en interrupciones que puede ser inhibido durante algún instante de tiempo.

5.2.2. Control de requisitos temporales

En el ámbito de la programación de sistemas de tiempo real, el control de requisitos temporales es esencial para garantizar un adecuado funcionamiento del sistema. Una de las restricciones más simples que puede tener un sistema empotrado es identificar y actuar en consecuencia cuando un determinado evento no tiene lugar.

Por ejemplo, considere que un sensor de temperatura ha de realizar una comprobación de la misma cada segundo. Sin embargo, y debido a cualquier tipo de problema, esta comprobación puede fallar y no realizarse en un determinado intervalo de tiempo. En este contexto, el sistema podría tener definido un **tiempo de espera límite** o **timeout** de manera que, si éste expira, el sistema pueda emitir una situación de fallo.

Otro posible ejemplo consistiría en garantizar que un determinado fragmento de código no tarde en ejecutarse más de un tiempo predefinido. De no ser así, el programa debería ser capaz de lanzar un mecanismo de recuperación de errores o una excepción.

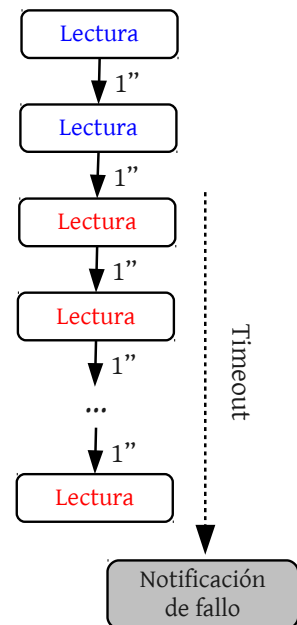


Figura 5.4: Uso de un *timeout* para identificar un fallo de funcionamiento en un sistema empotrado.



Recuerde que cuando un proceso está ejecutando una sección crítica, es bastante probable que otros procesos se encuentren bloqueados para poder acceder a la misma. En determinadas situaciones, será deseable controlar el tiempo máximo que un proceso puede estar en una sección crítica.

Hasta ahora, y dentro del contexto de la programación concurrente, no se ha controlado el tiempo empleado por un proceso o hilo en acceder, de manera exclusiva, a la **sección crítica**. En otras palabras, aunque un proceso obtenga el acceso exclusivo a una determinada sección de código, es deseable controlar el tiempo máximo que puede invertir en dicha sección. En esencia, esta gestión está asociada a un control de los requisitos temporales.

POSIX considera en algunas de sus primitivas un tiempo límite para acceder a la sección crítica. Por ejemplo, la primitiva de POSIX *pthread_cond_timedwait* devuelve un error si el tiempo absoluto indicado en uno de sus argumentos se alcanza antes de que un hilo haya realizado un *signal* o *broadcast* sobre la variable de condición asociada (vea sección 4.3.1).

Listado 5.8: Primitiva *pthread_cond_timedwait*.

```
1 #include <pthread.h>
2
3 int pthread_cond_timedwait(pthread_cond_t *restrict cond,
4                             pthread_mutex_t *restrict mutex,
5                             const struct timespec *restrict abstime);
6
7 int pthread_cond_wait(pthread_cond_t *restrict cond,
8                       pthread_mutex_t *restrict mutex);
```

Desafortunadamente, este tipo de bloqueo no es fácilmente limitable ya que mantiene una fuerte dependencia con la aplicación en cuestión. Por ejemplo, en el problema del buffer limitado se puede dar la situación de que los productores se bloqueen a la espera de que los consumidores obtengan algún elemento del buffer. Sin embargo, puede ser que éstos no estén preparadas hasta más adelante para obtener información. En este contexto, el productor debería limitar el tiempo de espera asociado, por ejemplo, al decremento del valor de un semáforo.

El siguiente listado de código muestra cómo hacer uso de la primitiva *sem_timedwait* de POSIX para tratar esta problemática.

Listado 5.9: Limitando el tiempo de bloqueo en un semáforo.

```
1 if (sem_timedwait (empty, tiempo) < 0) {
2     if (errno == ETIMEDOUT) {
3         // Tiempo cumplido.
4     }
5     else {
6         // Otro error.
7     }
8     else {
9         // Decremento realizado.
10 }
```

Otro concepto importante en el ámbito del control de requisitos temporales es el concepto de **evento disparador**, el cual permite abortar una acción si dicho evento ocurre antes. Típicamente, el evento disparador estará asociado al paso de un incremento de tiempo.

El siguiente listado de código muestra un ejemplo en Ada que limita el tiempo máximo de ejecución de una tarea a 0.25 segundos mediante un evento disparador.

Listado 5.10: Ejemplo de evento disparador en Ada.

```
1 select
2     delay 0.25;
3 then abort
4     TareaA;
5 end select;
```

Este tipo de esquemas permite capturar *código descontrolado*, acortando superiormente el tiempo de ejecución que puede consumir.



Recuerde que los tiempos límite de espera se suelen asociar con condiciones de error.

Finalmente, y antes de pasar a la siguiente sección en la que se discutirán distintas opciones de planificación en tiempo real, es importante destacar, como principal conclusión, que en ocasiones no es suficiente con que el software sea lógicamente correcto. Además, éste ha de satisfacer ciertas restricciones temporales, es decir, es necesario contemplar el contexto temporal en el que se ejecuta un sistema.

5.3. Esquemas de planificación

Como ya se introdujo anteriormente en la sección 5.1, en un sistema concurrente no existe, generalmente, una necesidad real de especificar el orden exacto en el que se ejecutan los distintos procesos que participan en el mismo.

Hasta ahora, las restricciones de orden vinculadas a las necesidades de sincronización de los problemas discutidos se han modelado a través del uso de semáforos, monitores o incluso colas de mensajes. Por ejemplo, si recuerda el problema de la barbería discutido en la sección 2.3.5, el cliente tenía que *despertar* al barbero antes de que éste efectuara la acción de *cortar*. Esta interacción se llevaba a cabo mediante un *signal* sobre el semáforo correspondiente.

No obstante, el **comportamiento global** de la simulación de la barbería no era determinista. Por ejemplo, ante distintas ejecuciones con el mismo número inicial de clientes, el resultado sería distinto debido a que el orden de ejecución de las distintas acciones que conforman cada proceso variaría de una simulación con respecto a otra.

Desde una perspectiva general, un sistema concurrente formado por n procesos tendrá $n!$ formas distintas de ejecutarse en un procesador, suponiendo un esquema no apropiativo. En esencia, y aunque el resultado final sea el mismo, el comportamiento temporal de los distintos procesos varía considerablemente [3]. Así, si uno de los procesos tiene impuesto un requisito temporal, entonces solamente lo cumplirá en aquellos casos en los que se ejecute al principio.

Un sistema de tiempo real necesita controlar este *no determinismo* a través de algún mecanismo que garantice que los distintos procesos ejecutan sus tareas en el plazo y con las restricciones previamente definidas. Este mecanismo se conoce tradicionalmente como **planificación** o **scheduling**.



La planificación estática es la que se basa en realizar una predicción antes de la ejecución del sistema, mientras que la dinámica se basa en tomar decisiones en tiempo de ejecución.

En un esquema de planificación se pueden distinguir dos elementos relevantes [3]:

1. Un algoritmo para ordenar el uso de los recursos del sistema, típicamente las CPU.
2. Un mecanismo para predecir el comportamiento del sistema cuando se aplica el algoritmo anterior. Normalmente, la predicción se basa en un planteamiento pesimista, es decir, considera el peor escenario posible para establecer una cota superior. Este tipo de mecanismos permiten conocer si los requisitos temporales establecidos para el sistema se cumplirán o no.

Conceptos relevantes en un esquema de planificación

Antes de llevar a cabo una discusión de distintas alternativas existentes para realizar la planificación de un sistema, resulta importante enumerar ciertos parámetros que son esenciales para acometer dicha tarea.

- **Tiempo máximo de ejecución (C)**, tiempo de ejecución de un proceso en el peor de los casos (*worst-case execution time* o WCET).
- **Interferencia (I)**, tiempo de interferencia del proceso, es decir, el tiempo que acumula un proceso como consecuencia de la interrupción de otros.
- **Bloqueo (B)**, tiempo de bloqueo del proceso en el peor caso (si es aplicable). Por ejemplo, considere el tiempo que un proceso se bloquea como consecuencia de una llamada a *wait* sobre un semáforo.
- **Deadline (D)**, tiempo límite de ejecución para un proceso, es decir, el tiempo máximo que pueda transcurrir antes de que finalice su ejecución.
- **Tiempo de respuesta (R)**, tiempo de respuesta de un proceso en el peor caso (si es aplicable).
- **Periodo (T)**, tiempo mínimo entre dos ejecuciones del proceso.
- **Prioridad (P)**, valor número que denota la importancia de un proceso.

Por otra parte, las tareas se pueden clasificar en **periódicas**, si se ejecutan con un periodo determinado, o **esporádicas**, si se ejecutan atendiendo a un evento temporal que las activa.



Como se discutirá posteriormente, en un sistema de tiempo real el objetivo principal consiste en garantizar que los tiempos de respuesta de todas las tareas sean menores a sus respectivos *deadlines*. Por el contrario, en un sistema que no es de tiempo real, uno de los objetivos principales consiste en minimizar el tiempo de respuesta de las tareas involucradas.

5.3.1. Modelo simple de tareas

En esta sección se describen los fundamentos de un modelo simple de tareas [3] que sirva como base para discutir los aspectos más importante de un sistema de planificación. Sobre este modelo se irán añadiendo distintas características que tendrán como resultado variaciones en el esquema de planificación inicial.

Las principales características de este modelo inicial son las siguientes:

- El número de procesos involucrados en la planificación es fijo.
- Todos los procesos son periódicos y su periodo (T) se conoce a priori.

- Todos los procesos son independientes entre sí.
- No se consideran tiempos asociados a eventos como cambios de contexto.
- El *deadline* (D) de todos los procesos es igual a su periodo. En otras palabras, todo proceso ha de completarse antes de volver a ejecutarse de nuevo.
- El esquema de asignación de prioridades es estático.
- Es posible desalojar a una tarea si existe otra con mayor prioridad lista para ejecutarse.
- El tiempo máximo de ejecución de cada tarea (C) es conocido.



La independencia de los procesos en el modelo simple posibilita que todos se ejecuten en paralelo, llevando al sistema a su carga máximo y generando un *instante crítico*.

5.3.2. El ejecutivo cíclico

El primer esquema de planificación a discutir se denomina ejecutivo cíclico debido al mecanismo utilizado para ejecutar las distintas tareas involucradas en la planificación. En esencia, el objetivo a alcanzar consiste en definir un modelo de planificación para un conjunto de tareas que se pueda repetir a lo largo del tiempo, de manera que todas ellas se ejecuten con una tasa apropiada.

Cuando se hace uso de este esquema no existe noción alguna de concurrencia, ya que las tareas son procedimientos que se planifican para que cumplan sus respectivos *deadlines*. Típicamente, el ejecutivo cíclico se modela como una tabla de llamadas a procedimientos.

Esta tabla representa el concepto de **hiperperiodo** o **ciclo principal**, dividido a su vez en una serie de **ciclos secundarios** de duración fija. El cálculo del ciclo principal (T_m) se realiza obteniendo el mínimo común múltiplo de los periodos de todos los procesos, es decir,

$$T_m = m.c.m(T_i, i \in \{1, 2, \dots, n\}) \quad (5.1)$$

mientras que el cálculo del ciclo secundario (T_s) se realiza mediante aproximación con el menor periodo de todos los procesos a planificar.

La planificación mediante ejecutivo cíclico se basa en *colocar* en cada ciclo secundario las tareas para garantizar que cumplan sus *deadlines*. La figura 5.5 muestra de manera gráfica el resultado de planificar el conjunto de procesos que contiene la tabla 5.1.

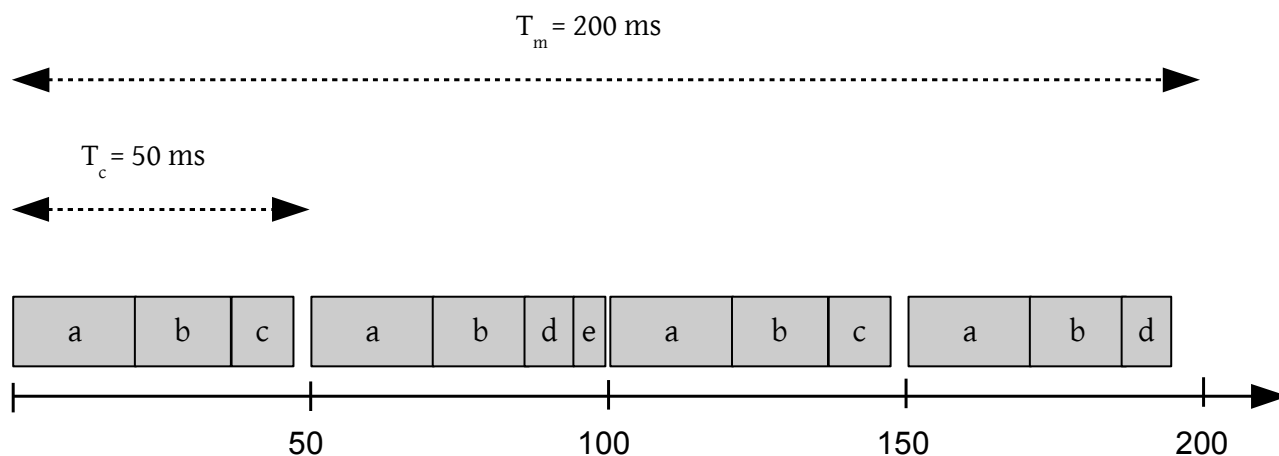


Figura 5.5: Esquema gráfico de planificación según el enfoque de ejecutivo cíclico para el conjunto de procesos de la tabla 5.1.

Proceso	Periodo (T)	Tiempo de ejecución (C)
a	50	20
b	50	16
c	100	10
d	100	8
e	200	4

Tabla 5.1: Datos de un conjunto de procesos para llevar a cabo la planificación en base a un ejecutivo cíclico.

Como se puede apreciar, el esquema de planificación es simple y consiste en ir asignando los procesos a los distintos *slots* temporales que vienen determinados por el valor del ciclo secundario. En este ejemplo en concreto, el valor del ciclo principal es de 200 ms (mínimo común múltiplo de los procesos), mientras que el valor del ciclo secundario es de 50 ms (menor periodo de un proceso).

Note cómo el esquema de planificación garantiza que todos los procesos cumplan sus *deadlines*. Además, la planificación de un ciclo completo (200 ms) garantiza que el esquema es completo en sucesivos hiperperiodos (en sucesivos bloques de 200 ms), ya que la ejecución de los procesos no variará. Desde el punto de vista del funcionamiento interno, se producirá una interrupción de reloj cada 50 ms, de manera que el planificador realiza rondas entre los cuatro ciclos secundarios que componen el hiperperiodo.

El uso de un enfoque de ejecutivo cíclico presenta **consecuencias relevantes** asociadas a las características inherentes al mismo [3]:

- Cada ciclo secundario es una secuencia de llamadas a procedimientos (los procesos no existen en tiempo de ejecución), como se muestra en la figura 5.6.

- No existe la necesidad de gestionar el acceso exclusivo a recursos, ya que los procesos nunca se ejecutan en paralelo.
- Los periodos de todos los procesos han de ser múltiplo del ciclo secundario.

```
do {  
    interrupción; llamada_a; llamada_b; llamada_c;  
    interrupción; llamada_a; llamada_b; llamada_d; llamada_e;  
    interrupción; llamada_a; llamada_b; llamada_c;  
    interrupción; llamada_a; llamada_b; llamada_d;  
while (1);
```

Figura 5.6: El código asociado al ejecutivo cíclico consiste simplemente en un bucle con llamadas a procedimientos en un determinado orden.

Precisamente, esta última consecuencia es la desencadenante de gran parte de las **limitaciones** de este enfoque:

- La integración de procesos no periódicos, es decir, esporádicos, no es sencilla.
- Los procesos con periodos elevados causan problemas de planificación.
- La construcción del ejecutivo cíclico es tediosa, sobre todo si el número de procesos es significativo.
- La integración de procesos con un tiempo de ejecución elevado es problemática y propensa a errores, debido a la necesidad de fragmentarlos en unidades de ejecución.

Aunque cuando se usa el ejecutivo cíclico no es necesario ningún test de planificabilidad adicional, su **construcción es problemática**, especialmente en sistemas de alta utilización. En términos de complejidad computacional, dicha construcción es similar a problemas de optimización clásicos, como por ejemplo el problema de la mochila. Este tipo de problemas son NP-completos y tienen una complejidad exponencial, siendo irresolubles para problemas de un cierto tamaño. En sistemas reales, es posible encontrarse con 50 ciclos secundarios y unas 500 entradas en la tabla del ejecutivo, por lo que es necesario hacer uso de heurísticas que obtengan *buenas soluciones* para construir el propio ejecutivo.

En conclusión, el ejecutivo cíclico puede ser una alternativa viable para sistemas con un número determinado de procesos periódicos. Sin embargo, su baja flexibilidad hace que los sistemas basados en la planificación de procesos, como los que se discuten en la siguiente sección, sean más adecuados.

5.3.3. Planificación basada en procesos

La planificación mediante ejecutivo cíclico no contempla la ejecución de procesos o hilos durante la ejecución, es decir, no hay una noción de concurrencia. Un planteamiento totalmente diferente consiste en soportar la ejecución de procesos, al igual que ocurre en los sistemas operativos modernos. En este contexto, la respuesta a responder es *¿Cuál es el proceso a ejecutar en cada momento?*.

En base a este planteamiento, un proceso puede estar en ejecución o esperando la ocurrencia de algún tipo de evento en un estado de suspensión. En el caso de procesos periódicos, este evento estará temporizado, mientras que en el caso de procesos no periódicos o esporádicos, dicho evento no estará temporizado.

Aunque existe una gran variedad de alternativas de planificación, en el presente capítulo se contemplarán las dos siguientes:

1. **Fixed Priority Scheduling (FPS)**, planificación por prioridad estática.
2. **Earliest Deadline First (EDF)**, planificación por tiempo límite más cercano.

El esquema FPS es uno de los más utilizados y tiene su base en calcular, antes de la ejecución, la prioridad P_i de un proceso. Dicha prioridad es fija y estática. Así, una vez calculada la prioridad de todos los procesos, en función de un determinado criterio, es posible establecer una ejecución por orden de prioridad. En sistemas de tiempo real, este criterio se define en base a los requisitos de temporización.

Por otra parte, el esquema EDF se basa en que el próximo proceso a ejecutar es aquél que tenga un menor *deadline* D_i absoluto. Normalmente, el *deadline* relativo se conoce a priori (por ejemplo, 50 ms después del comienzo de la ejecución del proceso), pero el absoluto se calcula en tiempo de ejecución, concediendo una naturaleza dinámica a este tipo de esquema de planificación.

Antes de pasar a discutir los aspectos relevantes de un planificador, es importante considerar la noción de **apropiación** en el contexto de una planificación basada en prioridades. Recuerde que, cuando se utiliza un esquema apropiativo, es posible que un proceso en ejecución con una baja prioridad sea desalojado por otro proceso de más alta prioridad, debido a que este último ha de ejecutarse con cierta urgencia. Típicamente, los esquemas apropiativos suelen ser preferibles debido a que posibilitan la ejecución *inmediata* de los procesos de alta prioridad.



Existen modelos de apropiación híbridos que permiten que un proceso de baja prioridad no sea desalojado por otro de alta hasta que no transcurra un tiempo limitado, como el modelo de apropiación diferida.

5.4. Aspectos relevantes de un planificador

Como ya se ha comentado anteriormente, el cometido principal de un planificador consiste en determinar en cada instante qué tarea, de aquellas listas para ejecutarse, se ejecutará en la CPU. Para ello, y desde un punto de vista general, el planificador es el responsable de gestionar los siguientes elementos:

- Un **sistema de asignación de prioridades**, responsable de establecer la prioridad de los procesos en base a algún criterio (normalmente asociado a requisitos temporales).
- Un **modelo de análisis del tiempo de respuesta**, responsable de calcular dicho tiempo para determinar si el sistema es planificable de acuerdo a las restricciones temporales impuestas por el entorno.



Un sistema de tiempo real es planificable si los tiempos de respuesta de todos los procesos o tareas que lo componen son menores que sus respectivos *deadlines*.

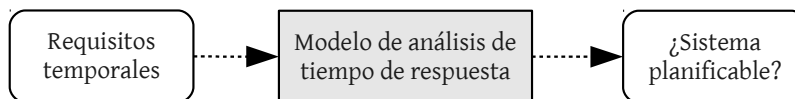


Figura 5.7: El principal cometido de un modelo de análisis del tiempo de respuesta es determinar si un sistema es planificable en base a sus requisitos temporales.

La implementación de este modelo de análisis se puede realizar de diversas formas, como por ejemplo mediante i) el cálculo del denominado **factor de utilización** que, aunque no es exacto, es simple y práctico, ii) la generación de un **esquema gráfico** que muestre la planificación del sistema o iii) utilizando algún esquema matemático para el **cálculo del tiempo de respuesta**.

En las siguientes secciones se discutirán aspectos tanto del sistema de asignación de prioridades como de cada una de las opciones mencionadas para implementar el modelo de análisis del tiempo de respuesta.

5.4.1. Sistema de asignación de prioridades

Considerando el modelo simple introducido en la sección 5.3.1, es posible establecer un esquema óptimo de asignación de prioridades denominado **rate monotonic** (*tasa monotónica*) [3]. Básicamente, a

cada proceso p_i se le asigna una prioridad única P_i basada en el valor de su periodo T_i , de manera que los procesos con un periodo menor tendrán una prioridad más alta,

$$\forall i, j \in \{1, 2, \dots, n\} (i \neq j), T_i < T_j \Rightarrow P_i > P_j \quad (5.2)$$

Este esquema de asignación es óptimo para un conjunto de procesos planificado mediante un esquema de prioridades estático y apropiativo.

Note que en el modelo simple planteado anteriormente, el *deadline* de un proceso es igual a su periodo, es decir, $D = T$. Por lo tanto, este esquema de asignación de prioridades coincide con el esquema **Deadline Monotonic Scheduling (DMS)**. La principal consecuencia es que los procesos o tareas con un menor *deadline* tendrán una mayor prioridad. Este esquema es óptimo para sistemas de tiempo real críticos, asumiendo el modelo simple con tareas esporádicas y bloqueos.

5.4.2. Modelo de análisis del tiempo de respuesta

Factor de utilización

El concepto de *factor de utilización* está vinculado a un test de planificabilidad que, aunque no es exacto, es muy simple y práctico. La base de este test se fundamenta en que, considerando la utilización de un conjunto de procesos, empleando el esquema FPS, se puede garantizar un test de planificabilidad si se cumple la siguiente ecuación para todos los procesos [7]:

$$\sum_{i=1}^N \left(\frac{C_i}{T_i} \right) \leq N(2^{\frac{1}{N}} - 1) \quad (5.3)$$

El sumatorio de la parte izquierda de la ecuación representa la utilización total del conjunto de procesos. Este valor se compara con el *límite de utilización* que viene determinado por la parte derecha de la ecuación. La siguiente tabla muestra el límite de utilización para algunos valores de N (número de procesos).

N (nº de procesos)	Límite de utilización
1	1
2	0.828
3	0.78
4	0.757
5	0.743
10	0.718

Tabla 5.2: Límite de utilización para distintos valores del número de procesos (N) involucrados en la planificación.



En determinadas situaciones, los tests de planificabilidad no necesitan ser estrictamente precisos, sino que es suficiente con proporcionar una aproximación realista.

Para ejemplificar este sencillo test de planificabilidad, la siguiente tabla muestra 3 procesos con sus respectivos atributos.

Proceso	T	C	P	U (Factor de utilización)
a	50	10	1	0.2
b	40	10	2	0.25
c	20	5	3	0.25

Tabla 5.3: Límite de utilización para distintos valores del número de procesos (N) involucrados en la planificación.

En el ejemplo de la tabla 5.3, la utilización total es de 0,7, valor menor a 0,78 (ver tabla 5.2), por lo que el sistema es planificable.

Esquema gráfico

En la figura 5.5 ya se mostró un esquema gráfico con el objetivo de mostrar un enfoque de planificación basado en el ejecutivo cíclico. En concreto, dicho esquema gráfico se corresponde con un **diagrama de Gantt**.

Por otra parte, también es posible emplear un esquema gráfico basado en el uso de las denominadas **líneas de tiempo**. La figura 5.8 muestra estos dos esquemas en relación a los procesos de la tabla 5.3.

En el caso particular de las líneas de tiempo, es necesario plantearse hasta cuándo es necesario trazar dichas líneas de tiempo para garantizar que ningún proceso incumple su *deadline*. La clave reside en dibujar una línea temporal igual al tamaño del periodo más largo [7].



Si todos los procesos cumplen su primer *deadline* temporal, entonces lo cumplirán también en el futuro.

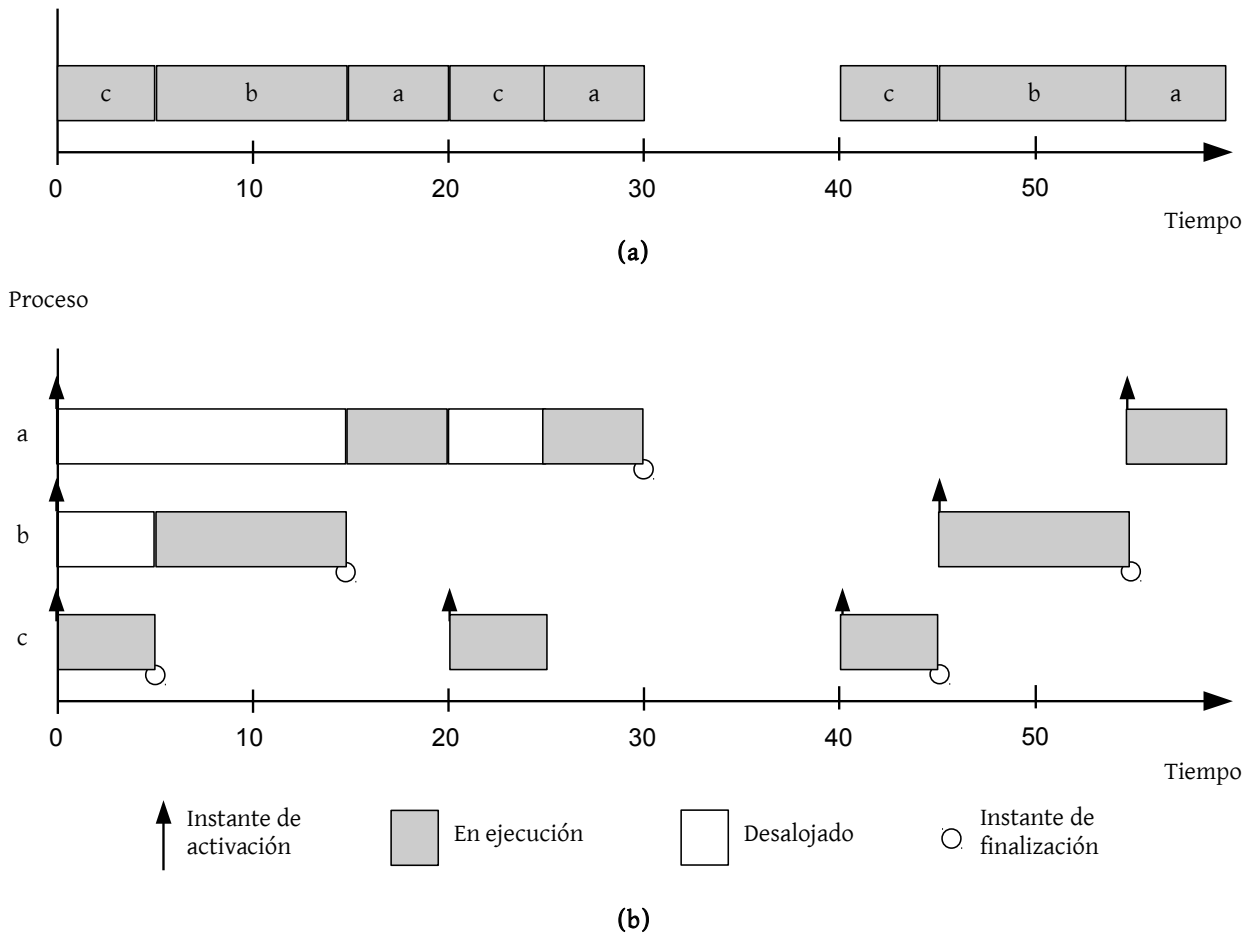


Figura 5.8: Distintos esquemas gráficos utilizados para mostrar patrones de ejecución para los procesos de la tabla 5.3. **(a)** Diagrama de Gantt. **(b)** Líneas de tiempo.

Cálculo del tiempo de respuesta

Aunque los tests basados en la utilización y los esquemas gráficos discutidos en las anteriores secciones son mecanismos sencillos y prácticos para determinar si un sistema es planificable, no son escalables a un número de procesos relevante y plantean ciertas limitaciones que hacen que sean poco flexibles. Por lo tanto, es necesario plantear algún otro esquema que sea más general y que posibilite añadir más complejidad al modelo simple de tareas introducido en la sección 5.3.1.

En esta sección se discute un esquema basado en **dos pasos** bien diferenciados:

1. Cálculo del tiempo de respuesta de todos los procesos mediante una aproximación analítica.

2. Comparación del tiempo de respuesta de todos los procesos con sus respectivos *deadlines*.

Estos dos pasos sirven para comprobar la expresión

$$R_i \leq D_i \forall i \in \{1, 2, \dots, n\} \quad (5.4)$$

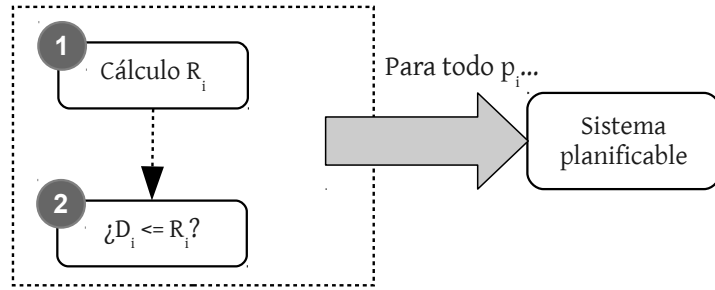


Figura 5.9: Esquema gráfico de un esquema de análisis del tiempo de respuesta para un conjunto de procesos o tareas.

es decir, si el tiempo de respuesta R_i de todo proceso p_i es menor a su *deadline* d_i , entonces se puede afirmar que el **sistema es planificable**. En otras palabras, se puede afirmar que todo proceso p_i cumplirá sus requisitos temporales. Para llevar a cabo estas comparaciones se realiza un análisis individual a nivel de proceso.

El caso más sencillo en un esquema apropiativo basado en prioridades fijas está representado por el proceso de mayor prioridad, ya que éste no recibirá ninguna interrupción por ningún otro proceso. En el peor de los casos, el tiempo de respuesta de dicho proceso será igual al tiempo de ejecución en el peor caso, es decir, $R = C$. Sin embargo, para cualquier otro proceso p_i que pueda recibir interrupciones por parte de procesos de mayor prioridad, la expresión del tiempo de respuesta R_i vendrá determinada por

$$R_i = C_i + I_i \quad (5.5)$$

donde I_i es el tiempo asociado a la interferencia máxima que p_i puede sufrir, como consecuencia de algún desalojo, en el intervalo de tiempo $[t, t + R_i]$.



La interferencia máxima sufrida por un proceso se da cuando todos los procesos que tienen una mayor prioridad se activan. Esta situación se denomina *instante crítico*.

A continuación se deducirá el tiempo de interferencia de un proceso p_i . Para ello, suponga que todos los procesos se activan en un

mismo instante de tiempo, por ejemplo en el instante 0, y que existe un proceso p_j con mayor prioridad que p_i , es decir, $P_j > P_i$. En este caso, el proceso p_j será activado un determinado número de veces (al menos una) dentro del intervalo $[0, R_i)$, es decir, dentro del intervalo en el que p_i completará su ejecución. Para obtener el número de veces que p_j se activa en dicho intervalo se puede emplear la ecuación

$$\text{Activaciones} = \lceil \frac{R_i}{T_j} \rceil \quad (5.6)$$

Informalmente, esta ecuación permite calcular el número de veces que el periodo de p_j es menor al tiempo de respuesta de p_i , es decir, el número de veces que p_j interrumpirá a p_i . La función techo se usa para obtener el menor entero que sea mayor que el resultado de dicha división. Por ejemplo, si R_i es 12 y T_j es 5, entonces se producirán tres activaciones de p_j en los instantes 0, 5 y 10.

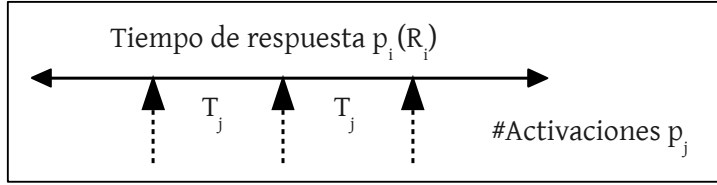


Figura 5.10: Esquema gráfico del número de activaciones de un proceso p_j con periodo T_j que interrumpe a un proceso p_i con un tiempo de respuesta total R_i .

A partir de este razonamiento, el cálculo del tiempo total de interrupción de p_j sobre p_i es sencillo, ya que sólo hay que considerar cuánto tiempo interrumpe p_j a p_i en cada activación. Dicho tiempo viene determinado por C_j , es decir, el tiempo de ejecución de p_j en el peor caso posible. Por lo tanto, la expresión de I_i es la siguiente:

$$I_i = \lceil \frac{R_i}{T_j} \rceil * C_j \quad (5.7)$$

En el ejemplo anterior, si $C_j = 7$, entonces I_i será igual a 21.

Debido a que cada proceso de más alta prioridad que p_i puede interrumpirlo (al igual que hace p_j), la expresión general de I_i es la siguiente:

$$I_i = \sum_{j \in hp(i)} (\lceil \frac{R_i}{T_j} \rceil * C_j) \quad (5.8)$$

donde $hp(i)$ representa el conjunto de procesos con mayor prioridad (*higher priority*) que p_i .

Si se sustituye el resultado de la ecuación 5.8 en la ecuación 5.5, entonces se obtiene la expresión final de R_i :

$$R_i = C_i + \sum_{j \in hp(i)} (\lceil \frac{R_i}{T_j} \rceil * C_j) \quad (5.9)$$

La ecuación 5.9 tiene la peculiaridad de que R_i aparece tanto en la parte izquierda como en la parte derecha de la misma. Sin embargo, dicha ecuación se puede resolver mediante una relación de recurrencia mediante un proceso de iteración:

$$\begin{aligned} w_i^0 &= C_i + \sum_{j \in hp(i)} (C_j) \\ w_i^1 &= C_i + \sum_{j \in hp(i)} (\lceil \frac{w_i^0}{T_j} \rceil * C_j) \\ w_i^2 &= C_i + \sum_{j \in hp(i)} (\lceil \frac{w_i^1}{T_j} \rceil * C_j) \end{aligned}$$

O en general:

$$w_i^n = C_i + \sum_{j \in hp(i)} (\lceil \frac{w_i^{n-1}}{T_j} \rceil * C_j) \quad (5.10)$$

Debido a que el conjunto $\{w_i^0, w_i^1, w_i^2, \dots, w_i^n\}$ es monótono no decreciente, en el ámbito de la planificación en tiempo real se podrá parar de iterar cuando se cumpla una de las dos siguientes condiciones:

- Si $w_i^{n-1} = w_i^n$, entonces ya se habrá encontrado la solución a la ecuación, es decir, el valor de R_i .
- Si $w_i^n > D_i$, entonces no es necesario seguir iterando ya que el sistema no será planificable respecto al proceso o tarea p_i , ya que su tiempo de respuesta es mayor que su *deadline*.

Una vez alcanzado el punto en el que se ha discutido un modelo analítico para el cálculo del tiempo de respuesta de un proceso, es importante recordar que **un sistema de tiempo real es planificable si y sólo si** $\forall i \in \{1, 2, \dots, n\}, R_i \leq D_i$, es decir, si los tiempos de respuesta de todos los procesos son menores o iguales a sus respectivos *deadlines*. Esta condición es una condición suficiente y necesaria para que el sistema sea planificable. Además, este modelo de análisis de tiempo real se puede completar, como se discutirá más adelante, siendo válido para computar el tiempo de respuesta.

A continuación se muestra un **ejemplo de cálculo del tiempo de respuesta** para el conjunto de procesos cuyas características se resumen en la tabla 5.4.

Proceso	T	C	P
a	80	40	1
b	40	10	2
c	20	5	3

Tabla 5.4: Periodo (**T**), tiempo de ejecución en el peor de los casos (**C**) y prioridad (**P**) de un conjunto de tres procesos para el cálculo del tiempo de respuesta (**R**).

El proceso de mayor prioridad, es decir, el proceso c , tendrá un tiempo de respuesta R_c igual a su tiempo de ejecución C_c , ya que no sufrirá interrupción alguna por otro proceso (el resto de procesos tienen menor prioridad).

Para el siguiente proceso de mayor prioridad, es decir, para el proceso b , será necesario realizar el cálculo de su tiempo de respuesta R_b utilizando la ecuación 5.10:

$$w_b^0 = C_b + C_c = 10 + 5 = 15$$

A continuación, se calcula w_b^1

$$w_b^1 = C_b + \sum_{j \in hp(b)} (\lceil \frac{w_b^0}{T_j} \rceil * C_j) = 10 + \lceil \frac{15}{40} \rceil * 5 = 15$$

Debido a que $w_b^0 = w_b^1$, no es necesario realizar otra iteración. Por lo tanto, $R_b = 15 \leq D_b = 40$ (recuerda que en el modelo simple, $T = D$), siendo planificable dentro del sistema.

El proceso a recibirá interrupciones de b y de c al ser el proceso de menor prioridad. De nuevo, se repite el procedimiento,

$$w_a^0 = C_a + C_b + C_c = 40 + 10 + 5 = 55$$

$$w_a^1 = C_a + \sum_{j \in hp(a)} (\lceil \frac{w_a^0}{T_j} \rceil * C_j) = 40 + \lceil \frac{55}{40} \rceil * 10 + \lceil \frac{55}{20} \rceil * 5 = 40 + 20 + 15 = 75$$

Debido a que $w_a^0 \neq w_a^1$, es necesario realizar otra iteración:

$$w_a^2 = C_a + \sum_{j \in hp(a)} (\lceil \frac{w_a^1}{T_j} \rceil * C_j) = 40 + \lceil \frac{75}{40} \rceil * 10 + \lceil \frac{75}{20} \rceil * 5 = 40 + 20 + 20 = 80$$

Finalmente

$$w_a^3 = C_a + \sum_{j \in hp(a)} (\lceil \frac{w_a^2}{T_j} \rceil * C_j) = 40 + \lceil \frac{80}{40} \rceil * 10 + \lceil \frac{80}{20} \rceil * 5 = 40 + 20 + 20 = 80$$

El tiempo de respuesta del proceso a cumple estrechamente su restricción temporal ya que, en este caso, $R_a = D_a$. Sin embargo, el sistema es planificable ya que los tiempos de respuesta de todos los procesos son menores o iguales a sus *deadlines*.

Es fácil comprobar que el sistema es planificable dibujando el diagrama de Gantt asociado al conjunto de procesos, tal y como muestra la figura 5.11.

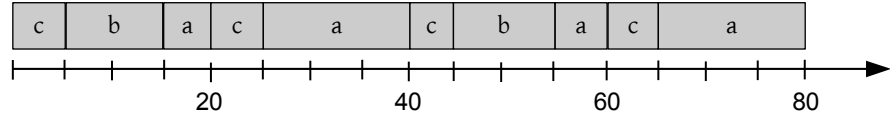


Figura 5.11: Diagrama de Gantt para el conjunto de procesos de la tabla 5.4.

5.4.3. Extendiendo el modelo simple de tareas

En esta sección se extiende el modelo simple planteado en la sección 5.3.1, incluyendo aspectos relevantes como por ejemplo tareas esporádicas y la posibilidad de que un proceso bloquee a otro como consecuencia, por ejemplo, de la adquisición de un recurso compartido.

En primer lugar, se consideran procesos con un **deadline inferior a su periodo** ($D < T$) con el objetivo de manejar un modelo menos pesimista que el anterior.

En segundo lugar, se considera la posibilidad de **interacción entre procesos**, hecho que condiciona el cálculo del tiempo de respuesta. Esta interacción se plantea en términos de un sistema concurrente de manera que un proceso de baja prioridad puede bloquear a otra tarea de alta prioridad debido a que utilice un recurso que esta última necesite durante un periodo de tiempo. Este tipo de situaciones se enmarcan en el concepto de **inversión de prioridad** y, aunque no se pueden evitar por completo, sí que es posible mitigar su efecto. Por ejemplo, se puede plantear un esquema que limite el bloqueo y que éste sea ponderable.

Si se considera este nuevo factor para realizar el cálculo del tiempo de respuesta de un proceso, entonces la ecuación 5.5 se reformula de la siguiente forma:

$$R_i = C_i + I_i + B_i \quad (5.11)$$

donde B_i es el máximo tiempo de bloqueo que puede sufrir el proceso p_i .

Como ejemplo práctico, considere un sistema con cuatro procesos (a , b , c y d) cuyas prioridades han sido calculadas en base a un esquema *Deadline Monotonic Scheduling*. El tiempo de activación de las tareas, así como su secuencia de ejecución, son conocidos. Además,

los procesos comparten dos recursos R y S de una única instancia (por ejemplo, dos impresoras), es decir, son recursos de acceso exclusivo. La tabla 5.5 muestra los datos de este ejemplo de inversión de prioridad.

Proceso	P	Instante de activación	Secuencia de ejecución
a	1	0	ERRRRE
b	2	2	EE
c	3	2	ESSE
d	4	4	EERSE

Tabla 5.5: Atributos relevantes para un ejemplo de planificación considerando inversión de prioridad (**E**, ciclo de ejecución; **R**, ciclo de acceso al recurso R ; **S**, ciclo de acceso al recurso S).

En la figura 5.12 se puede observar cómo se comporta el sistema en base a los atributos definidos en la tabla anterior.

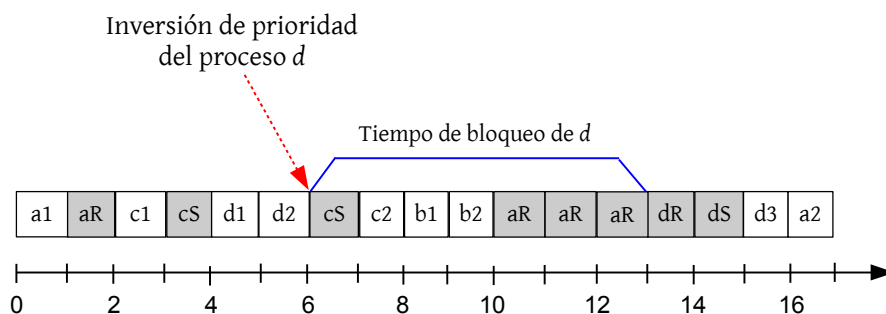


Figura 5.12: Diagrama de Gantt para el ejemplo de inversión de prioridad para el conjunto de procesos de la tabla 5.5.

A partir de la representación gráfica de la figura anterior, es sencillo obtener el tiempo de respuesta de los diferentes procesos del sistema: $R_a = 17$, $R_b = 8$, $R_c = 6$ y $R_d = 12$. En principio, era previsible esperar un tiempo de respuesta alto para el proceso de menor prioridad, es decir, para el proceso a , debido a que se ha utilizado un esquema apropiativo. Sin embargo, el tiempo de respuesta para el proceso de mayor prioridad, es decir, el proceso d , es excesivamente alto. La razón se debe a la **inversión de prioridad** que sufre en el ciclo 6 debido a la necesidad de utilizar un recurso (R) adquirido anteriormente por el proceso a , además de emplear un esquema basado en prioridades puramente estáticas.

El protocolo de herencia de prioridad

Un posible mecanismo para mitigar la inversión por prioridad consiste en utilizar la herencia de prioridad [4]. En esencia, este esquema se basa en que la prioridad de un proceso no sea estática, sino que ésta cambie cuando, por ejemplo, un proceso está a la espera de la

ejecución de una sección crítica por parte de otro proceso. Por ejemplo, si un proceso p_1 está suspendido esperando a que otro proceso p_2 termine de acceder a un objeto protegido, entonces la prioridad de p_2 obtiene el mismo valor (hereda) que la prioridad de p_1 , en caso de que fuera menor.

Este planteamiento posibilita que el proceso que herede una prioridad alta tenga preferencia sobre otros procesos que, en caso de no utilizar este esquema, alargarían el tiempo de respuesta de la tarea cuya prioridad se hereda. En el ejemplo de la figura 5.12, si el proceso a hubiese heredado la prioridad del proceso d al acceder al recurso R , entonces a tendría prioridad sobre b y c , respectivamente.



Cuando se utiliza un protocolo de herencia de prioridad, la prioridad de un proceso será el máximo de su propia prioridad y las prioridades de otros procesos que dependan del mismo.

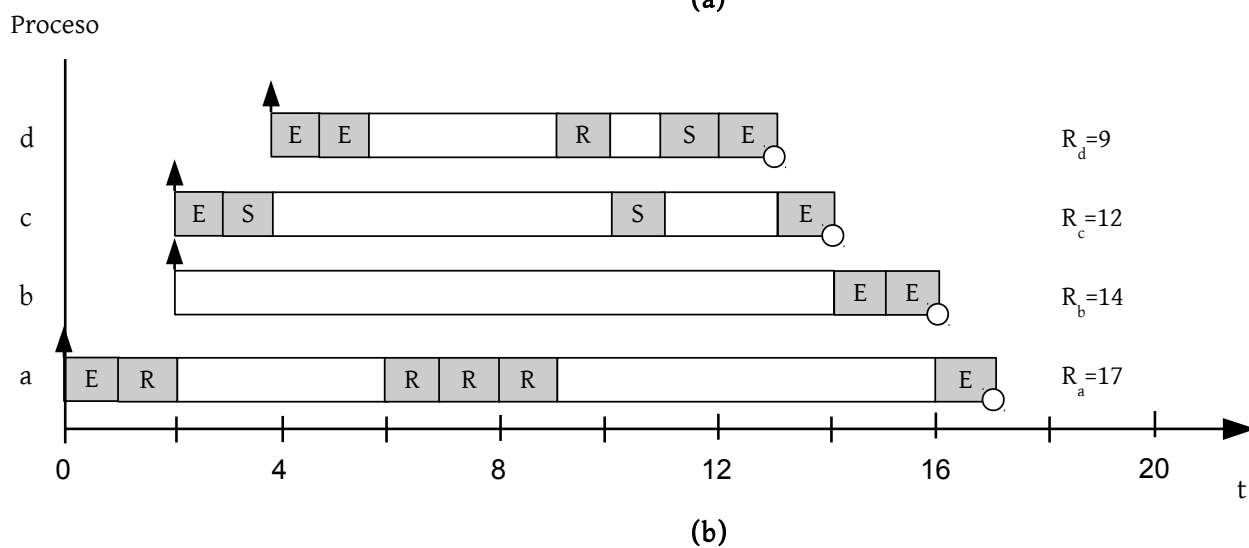
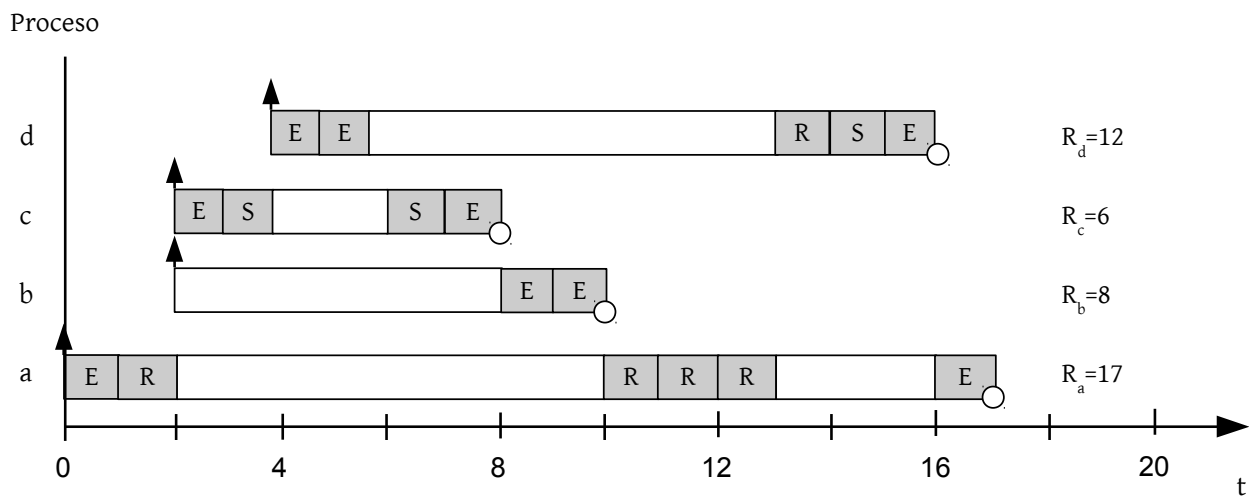
Para llevar a cabo un cálculo del tiempo de bloqueo de un proceso utilizando este esquema simple de herencia de prioridad se puede utilizar la siguiente fórmula [3]:

$$B_i = \sum_{r \in 1}^R (\text{utilizacion}(r, i) * C_r) \quad (5.12)$$

donde R representa el número de recursos o secciones críticas, mientras que $\text{utilizacion}(r, i) = 1$ si el recurso r se utiliza al menos por un proceso cuya prioridad es menor que la de p_i y, al menos, por un proceso con prioridad mayor o igual a p_i . Si no es así, $\text{utilizacion}(r, i) = 0$. Por otra parte, C_r es el tiempo de ejecución del recurso o sección crítica r en el peor caso posible.

La figura 5.13 muestra las líneas temporales y los tiempos de respuesta para el conjunto de procesos de la tabla 5.5 utilizando un esquema simple en el que se refleja el problema de la inversión de prioridad y el protocolo de herencia de prioridad, respectivamente.

En el caso del esquema simple, se puede apreciar de nuevo cómo el tiempo de respuesta del proceso d , el cual tiene la mayor prioridad, es de 17 unidades de tiempo, siendo el tiempo de respuesta más alto de todos los procesos. El extremo opuesto está representado, curiosamente, por el proceso de menor prioridad, es decir, el proceso a , cuyo tiempo de respuesta es el menor de todos (9 unidades de tiempo). Una vez más, se puede apreciar cómo el bloqueo del recurso S por parte del proceso a penaliza enormemente al proceso d .



↑ Instante de activación ■ En ejecución □ Desalojado ○ Instante de finalización

Figura 5.13: Líneas de tiempo para el conjunto de procesos de la tabla 5.5 utilizando **a)** un esquema de prioridades fijas con apropiación y **b)** el protocolo de herencia de prioridad. A la derecha de cada proceso se muestra el tiempo de respuesta asociado.

En el caso del protocolo de herencia de prioridad, este problema se limita considerablemente. Observe cómo el proceso a hereda la prioridad del proceso d cuando el primero adquiere el recurso R . Este planteamiento permite que a libere R tan pronto como sea posible para que d pueda continuar su ejecución.

De este modo, R_d se decrementa hasta las 9 unidades de tiempo, mientras que R_a se mantiene en 17. El proceso de mayor prioridad ha finalizado antes que en el caso con inversión de prioridad.

Desafortunadamente, R_c y R_b se han visto penalizados debido a la herencia de prioridad de a durante el tiempo de uso del recurso R .

El tiempo de bloqueo asociado al protocolo de herencia de prioridad se puede calcular fácilmente para el ejemplo discutido mediante la ecuación 5.12. Así, es posible obtener los respectivos tiempos de bloqueo de los diferentes procesos:

$$B_a = 0$$

$$B_b = C_{a,R} = 4$$

$$B_c = C_{a,R} = 4$$

$$B_d = C_{c,S} + C_{a,R} = 2 + 3 = 5$$

El protocolo de techo de prioridad inmediato

El protocolo de herencia de prioridad garantiza un límite superior en relación al número de bloqueos que un proceso de alta prioridad puede sufrir. Sin embargo, este límite puede contribuir a que el cálculo de tiempo de respuesta en el peor caso sea demasiado pesimista. Por ejemplo, en la sección anterior se ha mostrado cómo el protocolo de herencia de prioridad genera una **cadena de bloqueos** que hace que el tiempo de respuesta de varios procesos se vea afectado negativamente.

En este contexto, el gestor de recursos es el responsable de minimizar el bloqueo y eliminar las condiciones de fallo o interbloqueos. Los **protocolos de acotación de prioridad** [11] permiten tratar con esta problemática. En esta sección se discutirá el protocolo de techo de prioridad inmediato, pero también se comentarán aspectos del protocolo original de acotación de prioridad. Estos dos protocolos de acotación comparten algunas características (al ejecutarse en sistemas con una única CPU) [3]:

- Un proceso de prioridad alta sólo puede ser bloqueado una vez por procesos de prioridad inferior.
- Los interbloqueos se previenen.
- Los bloqueos transitivos se previenen.

- El acceso exclusivo a los recursos está asegurado.

Básicamente, los protocolos de acotación de prioridad garantizan que si un recurso está siendo bloqueado por un determinado proceso p_1 , y esta situación conduce a que otro proceso p_2 con mayor prioridad se bloquee, entonces no se permite que ningún otro recurso que pueda bloquear a p_2 sea bloqueado más que por p_1 .



En los protocolos de acotación de prioridad, la ejecución de un proceso se puede posponer incluso cuando el bloqueo de un recurso puede desencadenar un bloqueo múltiple de procesos de mayor prioridad.

En el caso particular del **protocolo de techo de prioridad inmediato** es importante considerar las siguientes propiedades [3]:

- Cada proceso p_i tiene asignada una prioridad P_i por defecto (por ejemplo, utilizando un esquema DMS).
- Cada recurso r_i tiene asociado un valor cota estático o **techo de prioridad** TP_i , que representa la prioridad máxima de los procesos que hacen uso del mismo.
- Cada proceso p_i puede tener una prioridad dinámica, que es el valor máximo entre su propia prioridad estática y los techos de prioridad de los recursos que tenga bloqueados.

En esencia, el protocolo de techo de prioridad inmediato consiste en que un proceso sólo puede hacer uso de un recurso si su prioridad dinámica es mayor que el techo de todos los recursos que están siendo usados por otros procesos. Por ejemplo, si un proceso p_1 utiliza un recurso r_1 con techo de prioridad Q y otro proceso p_2 con prioridad P_2 , $P_2 \leq Q$, quiere usar otro recurso r_2 , entonces el proceso p_2 también se bloquea.

La duración máxima de un bloqueo viene determinada por la siguiente ecuación:

$$B_i = \max\{C_{j,k}\}, j \in lp(i), k \in hc(i) \quad (5.13)$$

El **tiempo de bloqueo** B_i de un proceso p_i viene determinado por el máximo tiempo de uso de un recurso por parte de un proceso, considerando el conjunto de procesos con menor prioridad que P_i accediendo a recursos con un techo de prioridad igual o mayor a P_i .

El planteamiento del protocolo de techo de prioridad inmediato tiene ciertas consecuencias:

- Cada proceso se bloquea como máximo en una ocasión.

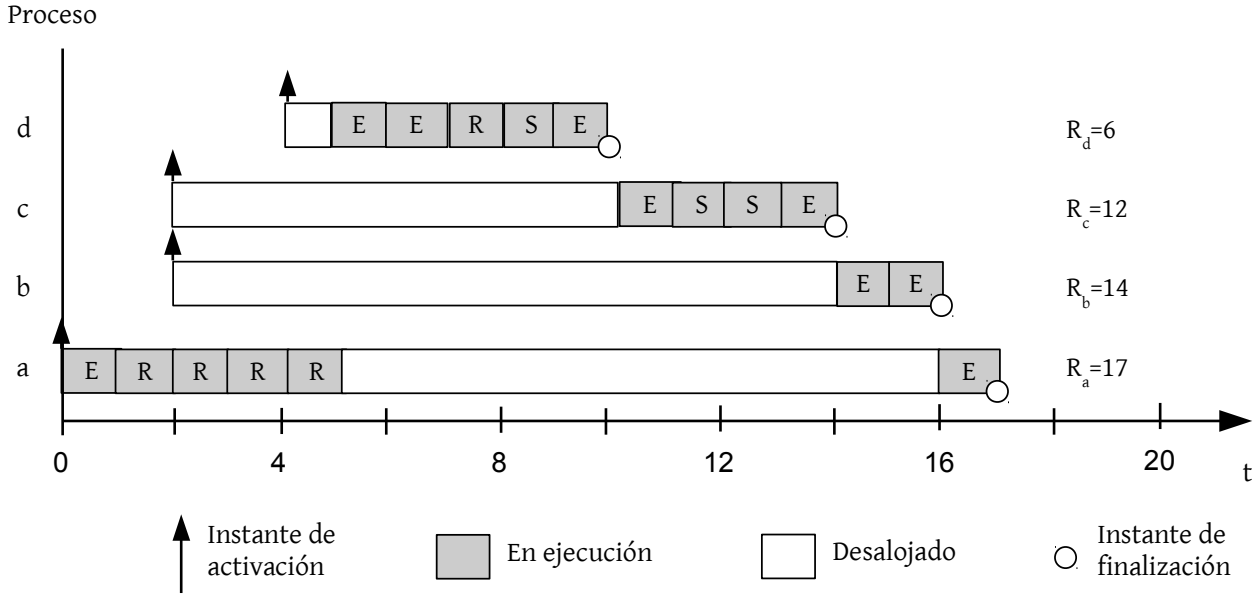


Figura 5.14: Línea de tiempo para el conjunto de procesos de la tabla 5.5 utilizando el protocolo de techo de prioridad inmediato.

- No existen cadenas de bloqueos ni interbloqueos.
- Cuando una tarea adquiere un recurso, entonces hereda su techo de prioridad mientras lo utilice.

La figura 5.14 muestra la línea de tiempo del conjunto de procesos de la tabla 5.5.

En primer lugar, es necesario calcular el techo de prioridad de los recursos R y S , considerando el máximo de las prioridades de los procesos que hacen uso de ellos:

$$TP_R = \max\{P_a, P_d\} = \max\{1, 4\} = 4$$

$$TP_S = \max\{P_c, P_d\} = \max\{3, 4\} = 4$$

En el instante de tiempo 1, el proceso a actualiza su prioridad dinámica a 4, es decir, utilizando el valor del techo de prioridad del recurso R . Por ello, y aun cuando en los instantes de tiempo 2 y 4, respectivamente, se activan el resto de procesos, el proceso a no es desalojado.

Es importante considerar cuándo se produce la reducción efectiva de la prioridad dinámica de un proceso. Por ejemplo, el proceso a retornará a su prioridad *por defecto* justo después de liberar el recurso R , es decir, en el instante de tiempo 5.

Por otra parte, el tiempo de respuesta de los procesos es idéntico al calculado cuando se utiliza el protocolo de herencia de prioridad, salvo

por el proceso d , el cual tiene la mayor prioridad. En este caso, dicho proceso ha reducido su tiempo de respuesta en un 33 %, pasando de 9 unidades de tiempo a 6.

Ejercicio práctico 1. Protocolo de herencia de prioridad

En este apartado se plantea un problema de planificación concreto en el que se aplica el protocolo de herencia de prioridad y se calculan tiempos de bloqueo, interferencia y respuesta.

Considere un sistema de tiempo real compuesto de cuatro tareas críticas cuyas características se muestran resumidas en la tabla 5.6. El método de planificación está basado en prioridades fijas con desalojo en base a un esquema *Deadline Monotonic Scheduling*. El acceso a los recursos se realiza mediante el protocolo de herencia de prioridad.

Tarea	T	C	D	r_1	r_2	r_3	r_4
a	80	10	80	3	-	-	5
b	150	20	150	2	2	1	-
c	100	10	15	1	1	-	-
d	500	12	30	2	-	4	-

Tabla 5.6: Periodo (T), tiempo de ejecución en el peor de los casos (C), *deadline*, y tiempo de uso de los recursos r_1 , r_2 , r_3 y r_4 para un conjunto de cuatro procesos.

Calcule las prioridades y el tiempo de bloqueo de las tareas. Obtenga el tiempo de respuesta de la tarea a.

En primer lugar, se procede a realizar el **cálculo de las prioridades** de las tareas en base a un esquema DMS, es decir, asignando una mayor prioridad a aquellas tareas cuyo *deadline* sea menor. Así,

$$P_a = 2, P_b = 1, P_c = 4, P_d = 3$$

En segundo lugar, el **tiempo de bloqueo** se obtiene haciendo uso de la ecuación 5.12. Así,

$$B_b = 0$$

$$B_a = C_{b,r_1} + C_{b,r_2} + C_{b,r_3} = 2 + 2 + 1 = 5$$

$$B_d = C_{b,r_1} + C_{a,r_1} + C_{b,r_2} + C_{b,r_3} = 2 + 3 + 2 + 1 = 8$$

$$B_c = C_{b,r_1} + C_{a,r_1} + C_{d,r_1} + C_{b,r_2} = 2 + 3 + 2 + 2 = 9$$

Recuerde que para contabilizar el tiempo de uso de un recurso r_i por parte de una tarea p_i , dicho recurso ha de usarse al menos por una tarea con menor prioridad que P_i y al menos por una tarea con prioridad mayor o igual a P_i .

Finalmente, para calcular el **tiempo de respuesta** de la tarea a es necesario hacer uso de la ecuación 5.11 (considerando no sólo el tiempo de bloqueo B_i sino también el de interferencia I_i) e iterando según la ecuación 5.10. Así,

$$w_a^0 = C_a + B_a + I_a = C_a + B_a + C_c + C_d = 10 + 5 + 10 + 12 = 37$$

$$\begin{aligned} w_a^1 &= C_a + B_a + I_a = C_a + B_a + \left\lceil \frac{w_a^0}{T_c} \right\rceil * C_c + \left\lceil \frac{w_a^0}{T_d} \right\rceil * C_d = \\ &10 + 5 + \left\lceil \frac{37}{100} \right\rceil * 10 + \left\lceil \frac{37}{50} \right\rceil * 12 = 37 \end{aligned}$$

Debido a que $w_a^0 = w_a^1$, entonces $R_a = w_a^1 = 37$.

Ejercicio práctico 2. Techo de prioridad inmediato

En este apartado se plantea un problema de planificación concreto en el que se aplica el protocolo de techo de prioridad inmediato y se calculan tiempos de bloqueo, interferencia y respuesta.

Considere un sistema de tiempo real compuesto de cuatro tareas críticas cuyas características se muestran resumidas en la tabla 5.7. El método de planificación está basado en prioridades fijas con desalojo y un esquema *Deadline Monotonic Scheduling*. El acceso a los recursos se realiza mediante el protocolo de techo de prioridad inmediato.

Tarea	T	C	D	r_1	r_2	r_3	r_4
a	100	20	100	3	-	-	5
b	200	20	200	2	2	-	-
c	100	5	15	1	-	-	-
d	50	12	20	2	-	4	-

Tabla 5.7: Periodo (**T**), tiempo de ejecución en el peor de los casos (**C**), *deadline*, y tiempo de uso de los recursos r_1 , r_2 , r_3 y r_4 para un conjunto de cuatro procesos.

Calcule las prioridades de las tareas y el techo de prioridad de los recursos. Obtenga el tiempo de bloqueo de las tareas y discuta si el sistema es planificable o no.

En primer lugar, se procede a realizar el **cálculo de las prioridades** de las tareas, considerando que la tarea con un menor *deadline* tiene una mayor prioridad. De este modo,

$$P_a = 2, P_b = 1, P_c = 4, P_d = 3$$

A continuación se obtiene el techo de prioridad de los recursos. Recuerde que el techo de prioridad de un recurso es la prioridad máxima de las tareas que hacen uso del mismo. Así,

$$TP_{r_1} = \max\{P_a, P_b, P_c, P_d\} = \max\{2, 1, 4, 3\} = 4$$

$$TP_{r_2} = \max\{P_b\} = \max\{1\} = 1$$

$$TP_{r_3} = \max\{P_d\} = \max\{3\} = 3$$

$$TP_{r_4} = \max\{P_a\} = \max\{4\} = 4$$

En segundo lugar, el **tiempo de bloqueo** se obtiene haciendo uso de la ecuación 5.13. Recuerde que dicho valor, cuando se aplica el protocolo de techo de prioridad inmediato, viene determinado por el máximo tiempo de uso de un recurso por parte un proceso, considerando el conjunto de procesos con menor prioridad que P_i accediendo a recursos con un techo de prioridad igual o mayor a P_i . Así,

$$B_d = \max\{C_{a,r_1}, C_{a,r_3}, C_{b,r_1}, C_{b,r_3}\} = \max\{3, -, 2, -\} = 3$$

A la hora de calcular B_d , Note cómo en la tabla 5.8 se resaltan los dos procesos (a y b) que cumplen la anterior restricción. Por lo tanto, habría que obtener el máximo de utilización de dichos procesos con respecto a los recursos que tienen un techo mayor o igual que P_d , tal y como muestra la tabla 5.9.

Tarea	T	C	D	r_1	r_2	r_3	r_4
a	100	20	100	3	-	-	5
b	200	20	200	2	2	-	-
c	100	5	15	1	-	-	-
d	50	12	20	2	-	4	-

Tabla 5.8: Obteniendo b_d (tiempo de bloqueo de la tarea d). Las tareas a y d son las que tienen menor prioridad que d (únicas que pueden bloquearla).

Tarea	T	C	D	r_1	r_2	r_3	r_4
a	100	20	100	3	-	-	5
b	200	20	200	2	2	-	-
c	100	5	15	1	-	-	-
d	50	12	20	2	-	4	-

Tabla 5.9: Obteniendo b_d (tiempo de bloqueo de la tarea d). Los recursos r_1 y r_3 tiene un techo de prioridad mayor o igual que la prioridad de d (las tareas de menos prioridad sólo pueden bloquear a d si acceden a estos recursos).

El resto de tiempos de bloqueo son los siguientes:

$$B_a = \max\{C_{b,r_1}\} = \max\{2\} = 2$$

$$B_b = \max\{0\} = 0$$

$$B_c = \max\{C_{a,r_1}, C_{b,r_1}, C_{d,r_1}\} = \max\{3, 3, 2\} = 3$$

Finalmente, para determinar si **el sistema es planificable o no** es necesario calcular el tiempo de respuesta de todas las tareas, haciendo uso de la ecuación 5.11 (considerando no sólo el tiempo de bloqueo B_i sino también el de interferencia I_i) e iterando según la ecuación 5.10. Para cada tarea p_i , habrá que comprobar si su tiempo de respuesta R_i es menor o igual a su *deadline* D_i . Así,

$$R_c = C_c + B_c + I_c = 5 + 3 + 0 = 8$$

debido a que p_c es la tarea de mayor prioridad y, por lo tanto, no recibe interferencias de otras tareas. Note que $R_c \leq D_c$ ($8 \leq 20$).

$$R_d = C_d + B_d + I_d$$

$$w_d^0 = C_d + B_d + C_c = 12 + 3 + 5 = 20$$

$$w_d^1 = C_d + B_d + \lceil \frac{w_d^0}{T_c} \rceil * C_c = 12 + 3 + \lceil \frac{20}{100} \rceil * 5 = 12 + 3 + 5 = 20$$

Debido a que $w_d^0 = w_d^1$, entonces $R_d = w_d^1 = 20$. Note que $R_d \leq D_d$ ($20 \leq 20$).

$$R_a = C_a + B_a + I_a$$

$$w_a^0 = C_a + B_a + C_d + C_c = 20 + 2 + 12 + 5 = 31$$

$$\begin{aligned} w_a^1 &= C_a + B_a + \lceil \frac{w_a^0}{T_d} \rceil * C_d + \lceil \frac{w_a^0}{T_c} \rceil * C_c = \\ &12 + 3 + \lceil \frac{31}{50} \rceil * 12 + \lceil \frac{31}{100} \rceil * 5 = 20 + 2 + 12 + 5 = 31 \end{aligned}$$

Debido a que $w_a^0 = w_a^1$, entonces $R_a = w_a^1 = 31$. Note que $R_a \leq D_a$ ($31 \leq 100$).

Finalmente,

$$R_b = C_b + B_b + I_b$$

$$w_b^0 = C_b + B_b + C_d + C_c + C_a = 20 + 0 + 20 + 12 + 5 = 57$$

$$\begin{aligned} w_b^1 &= C_b + B_b + \left\lceil \frac{w_b^0}{T_d} \right\rceil * C_d + \left\lceil \frac{w_b^0}{T_c} \right\rceil * C_c + \left\lceil \frac{w_b^0}{T_a} \right\rceil * C_a = \\ &20 + 0 + \left\lceil \frac{57}{50} \right\rceil * 12 + \left\lceil \frac{57}{100} \right\rceil * 5 + \left\lceil \frac{57}{100} \right\rceil * 20 = \\ &20 + 0 + 24 + 5 + 20 = 69 \end{aligned}$$

Como $w_b^0 \neq w_b^1$, es necesario seguir iterando.

$$\begin{aligned} w_b^2 &= C_b + B_b + \left\lceil \frac{w_b^1}{T_d} \right\rceil * C_d + \left\lceil \frac{w_b^1}{T_c} \right\rceil * C_c + \left\lceil \frac{w_b^1}{T_a} \right\rceil * C_a = \\ &20 + 0 + \left\lceil \frac{69}{50} \right\rceil * 12 + \left\lceil \frac{69}{100} \right\rceil * 5 + \left\lceil \frac{69}{100} \right\rceil * 20 = \\ &20 + 0 + 24 + 5 + 20 = 69 \end{aligned}$$

Debido a que $w_b^1 = w_b^2$, entonces $R_b = w_a^2 = 69$. Note que $R_b \leq D_b$ ($69 \leq 200$).

Una vez obtenidos todos los tiempos de respuesta de todos los procesos y comparados éstos con los respectivos *deadline*, se puede afirmar que **el sistema crítico es planificable**.

5.4.4. Consideraciones finales

El modelo simple de tareas discutido en este capítulo se puede extender aún más considerando **otros aspectos** con el objetivo de llevar a cabo un análisis más exhaustivo del tiempo de respuesta. Por ejemplo, sería posible considerar aspectos que tienen una cierta influencia en la planificación de un sistema de tiempo real, como por ejemplo el *jitter* o variabilidad temporal en el envío de señales digitales, los cambios de contexto, el tiempo empleado por el planificador para decidir cuál será el siguiente proceso o tarea a ejecutar, las interrupciones, etc.

Por otra parte, en el ámbito de los **sistemas de tiempo real no críticos** se pueden utilizar otras alternativas de planificación menos pesimistas y más flexibles, basadas en una variación más dinámica de las prioridades de los procesos y considerando especialmente la productividad del sistema.

En [3] se contemplan algunas de estas consideraciones y se realiza un análisis más exhaustivo de ciertos aspectos discutidos en el presente capítulo.

Apéndice **A**

El puente de un solo carril

A.1. Enunciado

Suponga un puente que tiene una carretera con un único carril por el que los coches pueden circular en un sentido o en otro. La anchura del carril hace imposible que dos coches puedan pasar de manera simultánea por el puente. El protocolo utilizado para atravesar el puente es el siguiente:

- Si no hay ningún coche circulando por el puente, entonces el primer coche en llegar cruzará el puente.
- Si un coche está atravesando el puente de norte a sur, entonces los coches que estén en el extremo norte del puente tendrán prioridad sobre los que vayan a cruzarlo desde el extremo sur.
- Del mismo modo, si un coche se encuentra cruzando de sur a norte, entonces los coches del extremo sur tendrán prioridad sobre los del norte.

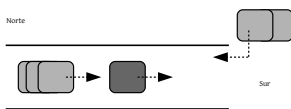


Figura A.1: Esquema gráfico del problema del puente de un solo carril.

A.2. Código fuente

Listado A.1: Makefile para la compilación automática

```
1 DIROBJ := obj/
2 DIREXE := exec/
3 DIRHEA := include/
4 DIRSRC := src/
5
6 CFLAGS := -I$(DIRHEA) -c -Wall
7 LDLIBS := -lpthread -lrt
8 CC := gcc
9
10 all : dirs manager coche
11
12 dirs:
13     mkdir -p $(DIROBJ) $(DIREXE)
14
15 manager: $(DIROBJ)manager.o $(DIROBJ)semaforoI.o $(DIROBJ)memoriaI.
16         o
17         $(CC) -o $(DIREXE)$@ $^ $(LDLIBS)
18
19 coche: $(DIROBJ)coche.o $(DIROBJ)semaforoI.o $(DIROBJ)memoriaI.o
20         $(CC) -o $(DIREXE)$@ $^ $(LDLIBS)
21
22 $(DIROBJ)%.o: $(DIRSRC)%.c
23     $(CC) $(CFLAGS) $^ -o $@
24
25 clean :
26     rm -rf *~ core $(DIROBJ) $(DIREXE) $(DIRHEA)*~ $(DIRSRC)*~
```

Listado A.2: Archivo semaforoI.h

```
1 #ifndef __SEMAFOROI_H__
2 #define __SEMAFOROI_H__
3 #include <semaphore.h>
4 // Crea un semáforo POSIX.
5 sem_t *crear_sem (const char *name, unsigned int valor);
6 // Obtiene un semáforo POSIX (ya existente).
7 sem_t *get_sem (const char *name);
8 // Cierra un semáforo POSIX.
9 void destruir_sem (const char *name);
10 // Incrementa el semáforo.
11 void signal_sem (sem_t *sem);
12 // Decrementa el semáforo.
13 void wait_sem (sem_t *sem);
14 #endif
```

Listado A.3: Archivo semaforoI.c

```
1 #include <stdio.h>
2 #include <errno.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <fcntl.h>
6 #include <semaforoI.h>
7
8 sem_t *crear_sem (const char *name, unsigned int valor) {
9     sem_t *sem;
10    sem = sem_open(name, O_CREAT, 0644, valor);
11    if (sem == SEM_FAILED) {
12        fprintf(stderr, "Error al crear el sem.: %s\n",
13                strerror(errno));
14        exit(1);
15    }
16    return sem;
17 }
18
19 sem_t *get_sem (const char *name) {
20    sem_t *sem;
21    sem = sem_open(name, O_RDWR);
22    if (sem == SEM_FAILED) {
23        fprintf(stderr, "Error al obtener el sem.: %s\n",
24                strerror(errno));
25        exit(1);
26    }
27    return sem;
28 }
29
30 void destruir_sem (const char *name) {
31    sem_t *sem = get_sem(name);
32    // Se cierra el sem.
33    if ((sem_close(sem)) == -1) {
34        fprintf(stderr, "Error al cerrar el sem.: %s\n",
35                strerror(errno));
36        exit(1);
37    }
38    // Se elimina el sem.
39    if ((sem_unlink(name)) == -1) {
40        fprintf(stderr, "Error al destruir el sem.: %s\n",
41                strerror(errno));
42        exit(1);
43    }
44 }
45
46 void signal_sem (sem_t *sem) {
47    if ((sem_post(sem)) == -1) {
48        fprintf(stderr, "Error al modificar el sem.: %s\n",
49                strerror(errno));
50        exit(1);
51    }
52 }
53
54 void wait_sem (sem_t *sem) {
55    if ((sem_wait(sem)) == -1) {
56        fprintf(stderr, "Error al modificar el sem.: %s\n",
57                strerror(errno));
58        exit(1);
59    }
60 }
```

Listado A.4: Archivo memoriaI.h

```

1 #ifndef __VARIABLEI_H__
2 #define __VARIABLEI_H__
3 // Crea un objeto de memoria compartida.
4 // Devuelve el descriptor de archivo.
5 int crear_var (const char *name, int valor);
6 // Obtiene el descriptor asociado a la variable.
7 int obtener_var (const char *name);
8 // Destruye el objeto de memoria compartida.
9 void destruir_var (const char *name);
10 // Modifica el valor del objeto de memoria compartida.
11 void modificar_var (int shm_fd, int valor);
12 // Devuelve el valor del objeto de memoria compartida.
13 void consultar_var (int shm_fd, int *valor);
14 #endif

```

Listado A.5: Archivo memoriaI.c (1 de 2)

```

1 #include <stdio.h>
2 #include <errno.h>
3 #include <string.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <fcntl.h>
7 #include <sys/mman.h>
8 #include <sys/stat.h>
9 #include <sys/types.h>
10
11 #include <memoriaI.h>
12
13 int crear_var (const char *name, int valor) {
14     int shm_fd;
15     int *p;
16
17     // Abre el objeto de memoria compartida.
18     shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
19     if (shm_fd == -1) {
20         fprintf(stderr, "Error al crear la variable: %s\n",
21             strerror(errno));
22         exit(1);
23     }
24
25     // Establecer el tamaño.
26     if (ftruncate(shm_fd, sizeof(int)) == -1) {
27         fprintf(stderr, "Error al trincar la variable: %s\n",
28             strerror(errno));
29         exit(1);
30     }
31
32     // Mapeo del objeto de memoria compartida.
33     p = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED,
34         shm_fd, 0);
35     if (p == MAP_FAILED) {
36         fprintf(stderr, "Error al mapear la variable: %s\n", strerror(
37             errno));
38         exit(1);
39     }
40
41     *p = valor;
42     munmap(p, sizeof(int));
43
44     return shm_fd;
45 }

```



```
44
45 int obtener_var (const char *name) {
46     int shm_fd;
47
48     // Abre el objeto de memoria compartida.
49     shm_fd = shm_open(name, O_RDWR, 0666);
50     if (shm_fd == -1) {
51         fprintf(stderr, "Error al obtener la variable: %s\n", strerror(
52             errno));
53         exit(1);
54     }
55     return shm_fd;
56 }
```

Listado A.6: Archivo memoriaI.c (2 de 2)

```

1  /* includes previos... */
2
3  #include <memoriaI.h>
4
5  void destruir_var (const char *name) {
6      int shm_fd = obtener_var(name);
7
8      if (close(shm_fd) == -1) {
9          fprintf(stderr, "Error al destruir la variable: %s\n",
10              strerror(errno));
11          exit(1);
12      }
13
14      if (shm_unlink(name) == -1) {
15          fprintf(stderr, "Error al destruir la variable: %s\n",
16              strerror(errno));
17          exit(1);
18      }
19  }
20
21 void modificar_var (int shm_fd, int valor) {
22     int *p;
23
24     // Mapeo del objeto de memoria compartida.
25     p = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
26         MAP_SHARED, shm_fd, 0);
27     if (p == MAP_FAILED) {
28         fprintf(stderr, "Error al mapear la variable: %s\n",
29             strerror(errno));
30         exit(1);
31     }
32
33     *p = valor;
34
35     munmap(p, sizeof(int));
36 }
37
38 void consultar_var (int shm_fd, int *valor) {
39     int *p, valor;
40
41     // Mapeo del objeto de memoria compartida.
42     p = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
43         MAP_SHARED, shm_fd, 0);
44     if (p == MAP_FAILED) {
45         fprintf(stderr, "Error al mapear la variable: %s\n",
46             strerror(errno));
47         exit(1);
48     }
49
50     *valor = *p;
51
52     munmap(p, sizeof(int));
53 }

```

Listado A.7: Proceso manager

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <signal.h>
6  #include <sys/wait.h>
7  #include <memoriaI.h>
8  #include <semaforoI.h>
9
10 #define COCHES 30
11 #define MAX_T_LANZAR 3
12 #define PUENTE "puente"
13 #define MUTEXN "mutex_n"
14 #define MUTEXS "mutex_s"
15 #define COCHESNORTE "coches_norte"
16 #define COCHESSUR "coches_sur"
17
18 pid_t pids[COCHES];
19 void liberarecursos (); void finalizarprocesos ();
20 void controlador (int senhal);
21
22 int main (int argc, char *argv[]) {
23     pid_t pid_hijo; int i;
24
25     srand((int) getpid());
26     // Creación de semáforos y segmentos de memoria compartida.
27     crear_sem(PUENTE, 1); crear_sem(MUTEXN, 1); crear_sem(MUTEXS, 1);
28     crear_var(COCHESNORTE, 0); crear_var(COCHESSUR, 0);
29
30     // Manejo de Ctrl+C.
31     if (signal(SIGINT, controlador) == SIG_ERR) {
32         fprintf(stderr, "Abrupt termination.\n"); exit(EXIT_FAILURE);
33     }
34
35     for (i = 0; i < COCHES; i++) { // Se lanzan los coches...
36         switch (pid_hijo = fork()) {
37             case 0:
38                 if ((i % 2) == 0) // Coche Norte.
39                     execl("./exec/coche", "coche", "N", PUENTE, MUTEXN, COCHESNORTE,
40                         NULL);
41                 else // Coche Sur.
42                     execl("./exec/coche", "coche", "S", PUENTE, MUTEXS, COCHESSUR,
43                         NULL);
44                 break;
45             } // Fin switch.
46             sleep(rand() % MAX_T_LANZAR);
47         } // Fin for.
48
49         for (i = 0; i < COCHES; i++) waitpid(pids[i], 0, 0);
50         liberarecursos(); return EXIT_SUCCESS;
51     }
52
53     void liberarecursos () {
54         destruir_sem(PUENTE); destruir_sem(MUTEXN); destruir_sem(MUTEXS);
55         destruir_var(COCHESNORTE); destruir_var(COCHESSUR);
56     }
57
58     void finalizarprocesos () {
59         int i; printf ("\n--- Finalizando procesos --- \n");
60         for (i = 0; i < COCHES; i++)
61             if (pids[i]) {
62                 printf ("Finalizando proceso [%d]... ", pids[i]);
63                 kill(pids[i], SIGINT); printf ("<Ok>\n");
64             }
65     }

```

```
63 }  
64  
65 void controlador (int senhal) {  
66     finalizarprocesos(); liberarecursos();  
67     printf ("\nFin del programa (Ctrol + C)\n"); exit(EXIT_SUCCESS);  
68 }
```
